Benchmarking Post-Quantum Key Encapsulation in S3-Compatible Object Storage under Client Concurrency

***Abstract*—Data encrypted today with classical public-key algorithms could be decrypted once large quantum computers exist. Cloud object stores keep data for years, so the client-side encryption that protects it must move to post-quantum algorithms such as ML-KEM, which adds a key-encapsulation step and a larger ciphertext to every stored object. So far, the cost of ML-KEM has been measured for the algorithm on its own and for securing network connections, but not for encrypting each object in a storage system that serves many clients at once. It has therefore been unclear whether ML-KEM slows such a system, under which conditions, and why. We address this problem by measuring per-object ML-KEM against classical key exchange on an S3-compatible object store, with two client implementations, two processor types and up to 500 concurrent workers, using pre-registered experiments and statistical equivalence tests. ML-KEM reduced throughput only when the client's processor was at or near full load, with one unexplained exception, and then by 4.0 to 4.9 % in our main client. Controlled experiments on one processor type traced most of that loss to the way the software library was called, and a second, native client showed that the size of the loss depends on the implementation. The lasting cost is storage: 1,056 more bytes per object than with classical key exchange. The results cover two clients, one single-node store and one cloud provider.**

***Index Terms*—ML-KEM, hybrid KEM, envelope encryption, client-side encryption, cloud storage security, HPKE, equivalence testing, performance evaluation**

I. Introduction

Cloud object stores keep data for years, and much of that data is encrypted under keys established with RSA or elliptic-curve Diffie–Hellman. Shor's algorithm would allow a sufficiently large quantum computer to recover those keys, so an adversary who records ciphertext today can decrypt it once such a machine exists, a threat usually called "harvest now, decrypt later". For data at rest the exposure lasts as long as the data is retained. In August 2024 NIST published FIPS 203, standardizing the module-lattice key-encapsulation mechanism ML-KEM [1].

With client-side envelope encryption, as implemented by the Amazon S3 Encryption Client and by Azure Blob Storage client libraries [2], [3], every object is encrypted under its own data key, and that key is protected for the recipient and stored with the object.

A public-key KEM per object is needed only in some deployments. If writer and reader share a symmetric key-encryption key, wrapping the data key with AES is already quantum-safe and no KEM is needed; that is our baseline B1. A public-key mechanism is required when the writer must not be able to decrypt, or does not share a secret with the reader: write-only producers such as log shippers, backup agents and sensors that hold only a public key; objects encrypted to several recipients or to an escrow key; and data handed across organizational boundaries. These deployments use RSA or elliptic-curve key establishment today, and they are the ones that must migrate. For them, ML-KEM places a key establishment and its 1,088-byte ciphertext on every PUT and every GET rather than once per connection.

What is known falls into three groups of measurements. Primitive-level studies report the cost of key generation, encapsulation and decapsulation and the sizes of keys and ciphertexts on several processor families [4], [5], [6], [7]. Studies of post-quantum TLS and KEMTLS measure connection establishment under latency, loss and bandwidth limits [8], [9], [10], [11]. Storage studies measure the S3 interface without cryptography [12] and client-side encryption without post-quantum key establishment [13], [14].

What is not known is the cost on the data path of an object store. None of these measures ML-KEM where an object store would use it: once per object, on a data path whose throughput is set jointly by client CPU, network, metadata handling and the storage server, with many clients at once. A back-of-envelope estimate already says that tens of microseconds of KEM against milliseconds of storage round trip should be invisible. What that estimate cannot say is whether it holds under concurrency, in which regime it fails, by how much, and whether the cause is the KEM computation or the larger ciphertext it carries through the data path (Section II, Table I).

The thesis of this paper is the following. In the configurations we measured, replacing X25519 by per-object ML-KEM lowered throughput only where the client was at or near CPU saturation, with one unexplained exception in the native reader. The size of the loss there depended on the client implementation: it ranged from no measurable loss (0.999 of X25519's throughput, the Python client calling liboqs through one foreign call) through 4.0–4.9 % (the same client through the standard binding) to 9.5 % (a native Go reader on one core). The cost that every implementation pays is storage: a larger ciphertext with every object. The observations that do not fit this picture are collected in Section V-G.

The contributions are the following.

C1 The saturation-regime result, with a method to apply it: measurements of ML-KEM-768 against X25519 over object size, 1–500 concurrent workers, two client archi-

tectures and two client implementations, with the independently restarted run as the experimental unit and pre-registered equivalence tests. We use a CPU accounting identity (Section IV-C) as a method: under client saturation the throughput ratio of two modes is the inverse ratio of their client CPU per request, which turns a migration question into a CPU measurement and a saturation check (Sections V-B–V-C).

C2 A binding artifact, and what it implies for library authors: two pre-registered controls on the x86 Python client show that its client-bound deficit is produced by a library binding that makes four foreign calls per decapsulation, and for 17 % by the larger manifest (Section V-C).

C3 The storage cost: the per-object metadata cost of ML-KEM and of two hybrids, separated into ciphertext and encoding, scaled to fleet sizes, and set against the 2 KB limit that Amazon S3 places on user-defined object metadata (Section V-E).

Secondary results are the measured trade-off of bounded session amortization, which saves a few percent of client CPU and changes throughput only on a client-bound workload (Section V-F); the attribution of the store's small-object write ceiling to its metadata path and not to its disk (Section V-B); and the observation that RSA-4096 is the one mode that moves the bottleneck in the Python client. The instrumented client, the harness, the native reader and the analysis pipeline are an artifact that supports these results and are not claimed as a contribution; neither is the architecture or commit protocol, which is not a new cryptographic construction (Table III). The open-loop capacity observations in the supplementary material describe the store, not post-quantum cryptography, and their main lesson is known [15].

The scope of the study is limited as follows. We do not evaluate the security of ML-KEM, side channels, or transport security, which is held constant. All performance statements concern one Python client stack and one small native reader, one single-node MinIO deployment and one cloud provider (Section VI). The native reader covers the GET path of three modes only. Section VI lists the experiments that remain.

This paper measures what per-object ML-KEM-768 costs against RSA-4096, X25519 and symmetric-only baselines on the same data path; how that cost depends on object size, concurrency and a hybrid construction, and what causes it; and what bounded sessions trade for the key-establishment work they save.

II. Background and Related Work

A. Post-Quantum KEMs, Hybrids and HPKE

A key-encapsulation mechanism (KEM) consists of key generation, encapsulation, which takes the recipient's public key and outputs a ciphertext and a shared secret, and decapsulation, which recovers the secret with the private key. ML-KEM, derived from CRYSTALS-Kyber, relies on the hardness of module learning with errors. ML-KEM-768 targets NIST security category 3 with a 1,184-byte encapsulation key, a 1,088-byte ciphertext and a 32-byte shared secret [1]. Decapsulation uses *implicit rejection*: a modified ciphertext yields a pseudorandom secret instead of an error, so tampering is detected by the authenticated encryption that follows.

A hybrid KEM combines a classical and a post-quantum component so that the result stays secure while either remains unbroken. The combiner matters: X-Wing [16], [17] combines X25519 with ML-KEM-768 as $\mathrm{SHA3\text{-}256}(ss_M \| ss_X \| ct_X \| pk_X \| label)$ and derives both private keys from one 32-byte seed; its security proof depends on these concrete choices and does not carry over to other ways of mixing two secrets.

Hybrid Public Key Encryption (HPKE, RFC 9180) [18] is the standard construction for encrypting to a public key: a KEM produces a shared secret, a key schedule derives an AEAD key and base nonce from it and from application-supplied context, and messages are sealed under sequence-numbered nonces. An IETF draft adds ML-KEM-768 and an X25519+ML-KEM-768 hybrid as HPKE KEMs [19]. Per-object envelope encryption has the structure of HPKE's single-recipient base mode. HPKE [18] and its post-quantum extension [19] specify how to encrypt to an ML-KEM or hybrid public key but contain no measurements; the file-encryption tool age adopted such recipients [20]. Our envelope predates our reading of that draft and is not HPKE-conformant; Section III maps one onto the other and states what transfers. For hybrids, the security of the result depends on the combiner. X-Wing [16], [17] comes with a proof for its concrete combiner, and the general theory of KEM combiners states the conditions under which a combination stays secure if one component fails [21], [22]. Our mode P2 concatenates two shared secrets into HKDF and has not been checked against those conditions, which is why it carries no security claim and serves only as a performance reference (Table III).

In envelope encryption a per-object data key encrypts the object, and key establishment protects that key. We derive data keys with HKDF-SHA256 [23] and encrypt with AES-256-GCM, whose security requires that a nonce is never reused under one key [24]. The classical alternatives are key transport with RSA-OAEP [25] and ephemeral-static X25519 [26]. Object stores expose buckets and keys through HTTP operations, chiefly PUT and GET of whole objects; the S3 interface is the de facto standard [12]. We use MinIO [27], compiled from its final community release (RELEASE.2025-10-15T17-29-55Z). Because that community edition is no longer developed, our store-specific results (ceilings, knee) should be read as those of one representative single-node S3 server, not as current product behaviour.

The closest analogue of a per-object KEM with bounded reuse is secure messaging. Signal's PQXDH adds a post-quantum KEM to initial key agreement [28], and Apple's PQ3 applies one to initial key establishment and to a periodic rekeying step as well [29]. There the unit of amortization is a session or a rekeying interval, and the question of how many messages one KEM output may protect is the same one our session mode poses for objects (Section V-F). We

are not aware of throughput measurements of these protocols that would transfer to an object store.

B. Related Measurements

Table I positions this study. The entries describe the scope of each work, not a shortcoming: a TLS study is not incomplete for omitting S3, but its results cannot establish the cost of key establishment in an object-store data path.

Existing measurements stop at two boundaries. At the primitive level, the Kyber design paper [32], the standard [1], library benchmarks [4] and recent comparative preprints [5], [6], [7] give the cost and sizes of the three KEM operations on several processor families; they establish that platform and implementation matter, which is why we repeat our primary comparison on a second architecture and test a second implementation, but they contain no storage, concurrency or end-to-end request latency. At the transport level, studies of post-quantum TLS and KEMTLS [8], [9], [10], [11], [33] measure connection establishment under latency, loss and bandwidth limits and decompose the handshake. They are the closest systems precedents, but their amortization boundary differs: a handshake protects a connection, whereas in client-side object encryption key establishment and its ciphertext recur for each object.

Storage studies measure the S3 interface without cryptography, and client-side encryption without post-quantum key establishment. Gadban and Kunkel [12] attribute losses on the S3 path to layers of the client stack, a separation our harness applies to cryptographic stages; Henziger and Carlsson [13] and Ozeri et al. [14] concern where client-side encryption sits and what it costs synchronization. Two 2026 preprints approach post-quantum cloud storage from the design side: Prikutse et al. [30] combine AES-256-GCM, X25519+ML-KEM-768 and ML-DSA-65 and amortize signatures over a batch of files, and Junaid [31] proposes a client-side hybrid architecture with self-hosted key management. According to their abstracts, neither evaluates concurrent S3 PUT/GET or compares with classical per-object key establishment; we have read only the abstracts. Bounded key reuse is established practice in this setting: Amazon S3 Bucket Keys derive per-object data keys from a time-limited bucket-level key and reduce key-service requests by up to 99 % [34], and the AWS Encryption SDK caches data keys under age, message and byte limits [35]. Our session mode applies that principle to ML-KEM roots and quantifies the trade-off.

In October 2026 we ran keyword searches, restricted to arXiv, dblp and Semantic Scholar and on the open web, combining *ML-KEM*, *Kyber* or *X-Wing* with *object storage*, *S3*, *MinIO*, *client-side encryption*, *envelope encryption* and *concurrency*; the queries and their outcome are logged in the artifact. This is a keyword search, not a systematic review. It returned primitive benchmarks, transport studies, vendor tutorials and the two storage designs above, but, to our knowledge, no measurement of per-object ML-KEM under concurrent S3 load. The search did not cover IEEE Xplore, the ACM Digital Library, the IACR ePrint archive or Google Scholar, which is a limitation of that statement.

C. Benchmarking Method

Three strands of method inform our design. Cloud instances show run-to-run and time-dependent performance variation large enough to change conclusions drawn from single runs [36], [37], [38], which is why our experimental unit is the independently restarted run, compared in randomized blocks and never across sessions. Reporting rules for performance measurements [39], [40] ask for intervals over independent repetitions and a full description of the setup. Schroeder et al. [15] show that closed- and open-loop load generators produce systematically different behaviour; our open-loop observations are an instance of that result. For equivalence we follow the two one-sided tests procedure [41], [42].

III. Design and Threat Model

The network and the object store are honest-but-curious for confidentiality and potentially malicious for integrity: an adversary may read, replace, delete, reorder or replay stored ciphertexts and manifests. The adversary does not hold the recipient's decapsulation key and may in the future operate a quantum computer. Availability, traffic analysis, deletion of whole objects, compromised endpoints and side channels in the client are out of scope.

Writers are not authenticated. The envelope encrypts to a public key and carries no signature, and neither does HPKE base mode. Integrity therefore means that an object written by an honest writer cannot be modified, reordered, truncated, extended or spliced without detection. It does not mean that an object is authentic: anyone who knows the recipient's public key, including the store, can create a new object under any identity, and it will decrypt without error. Sender authentication is out of scope.

Rollback is not prevented. Replay is detected only where it crosses an identity: a manifest or chunk moved to another object or another version fails authentication, because the object identity and version are bound into the key and into every chunk. Serving an *older, complete and valid version* of the same object is a different matter. A store that withholds or deletes the newest manifest makes the reader accept the previous version, and nothing in the design detects that. Freshness would need state outside the store, such as a signed version counter or a transparency log; it is out of scope.

A third limit concerns key commitment. AES-GCM is not key-committing: a ciphertext can be constructed that decrypts validly under more than one key [43], [44]. We accept this for the single-recipient setting studied here, on the following argument. Since writers are not authenticated, a malicious writer can already store any plaintext it likes under a key the recipient will derive; a ciphertext that also decrypts under a second key gives it no further capability against one recipient. That argument is not a proof and we make no formal claim (Table III). It does not extend to several recipients, where one

TABLE I
SCOPE OF RELATED WORK. ✓ = COVERED; – = OUTSIDE THE STUDY'S SCOPE OR NOT REPORTED. P = PEER-REVIEWED, A = PREPRINT (ARXIV).

Study	Type	Setting	PQ KEM	Hybrid KEM	Object store	Per-object key establishment	Concurrency sweep	Queueing analysis
Paquin et al. [8]	P	TLS 1.3 under emulated latency and loss	✓	✓	–	–	–	–
Sikeridis et al. [9]	P	Post-quantum authentication in TLS 1.3	–	–	–	–	–	–
Sosnowski et al. [10]	P	TLS 1.3 handshakes, black- and white-box	✓	✓	–	–	–	–
Giron et al. [11]	P	Hybrid KEMTLS, simulated and real networks	✓	✓	–	–	–	–
Barbosa et al. [16]	P	X-Wing construction and security proof	✓	✓	–	–	–	–
Rodriguez-Alvarez [5]	A	ML-KEM against RSA and ECC, primitives	✓	–	–	–	–	–
Demir et al. [6]	A	Kyber and Dilithium primitives	✓	–	–	–	–	–
Lopez et al. [7]	A	PQC on resource-constrained devices	✓	–	–	–	–	–
Gadban and Kunkel [12]	P	S3 interface for HPC workloads	–	–	✓	–	✓	–
Henziger and Carlsson [13]	P	Client-side encryption and delta encoding	–	–	–	–	–	–
Prikutse et al. [30]	A	Hybrid PQ model for multi-file cloud storage	✓	✓	–	–	–	–
Junaid [31]	A	Hybrid PQ client-side encryption, self-hosted keys	✓	✓	–	–	–	–
This work		Client-side envelope encryption on S3 (MinIO)	✓	✓	✓	✓	✓	✓

ciphertext could decrypt to different plaintexts for different recipients, or to password-derived keys: a multi-recipient design must use a key-committing construction [45].

Table II lists the modes. All encrypting modes share the HKDF and chunked AES-GCM path. They differ in the key-establishment computation *and* in the size of the manifest that carries its ciphertext (622 to 2,085 bytes), which changes parsing, hashing and bytes transferred. A difference between two modes is therefore the cost of key establishment together with the handling of its ciphertext; Section V-C separates the two as far as the data allows. B0 stores plaintext through the same framing; B1 derives keys from a pre-shared 32-byte key and is the symmetric-only floor; R1 transports a random secret with RSA-OAEP-4096; B2 uses ephemeral-static X25519; P1 uses ML-KEM-768. Two hybrids combine X25519 with ML-KEM-768. *P3 is X-Wing* as specified in [17]: we implement its key derivation from a 32-byte seed, its SHA3-256 combiner and its ciphertext layout, and verify key generation, encapsulation and decapsulation against the draft's published test vectors. *P2 is a study-specific concatenation hybrid*: it stores the ML-KEM ciphertext followed by the X25519 ephemeral key and feeds both 32-byte secrets, concatenated, to HKDF together with the object context. P2 predates our X-Wing implementation; we keep it as a performance reference and make no security claim for its combiner. N1 amortizes one ML-KEM encapsulation over a bounded session (described below). A reader must name the mode it expects, so a planted plaintext object cannot downgrade an encrypted one.

Algorithm 1 shows PUT and GET for mode P1. An object is identified by $I = (tenant, bucket, key, version)$, where versions are time-ordered UUIDv7 values. The context ctx is canonical JSON of I, the mode, the algorithm names, chunk size and count, plaintext size, recipient key identifier and the SHA-256 hash of the key-establishment ciphertext

TABLE II
KEY-ESTABLISHMENT MODES. SIZES IN BYTES; THE PER-OBJECT VALUE IS WHAT EVERY OBJECT'S MANIFEST CARRIES.

Mode	Key establishment	Hard problem	Public key	Per object	PQ
B0	none (plaintext)	–	–	–	–
B1	pre-shared key	–	–	0	yes
R1	RSA-OAEP-4096	factoring	512	512	no
B2	X25519	ECDLP	32	32	no
P1	ML-KEM-768	MLWE	1,184	1,088	yes
P2	X25519+ML-KEM-768, concatenation	both	1,216	1,120	yes
P3	X-Wing (X25519+ML-KEM-768)	both	1,216	1,120	yes
N1	ML-KEM-768 session	MLWE	1,184	1,088	yes

C; it is bound into the data key and into every chunk. Objects are split into 4 MiB chunks, and chunk j is encrypted with nonce $p \| \mathrm{be}_{64}(j)$ under a random 32-bit prefix p and associated data $ctx \| j \| \ell_j \| final_j$, so deletion, reordering, truncation, extension and cross-object splicing of chunks fail authentication. This is the counter-and-final-flag pattern of the STREAM construction [46], with the flag carried in the associated data instead of the nonce; we do not claim STREAM's proof for our variant. Each version is stored under `key.pqcs/version/` as a ciphertext stream `data.enc` and a manifest `manifest.json` holding ctx, C, the salt and the prefix. The manifest is written last and serves as the commit record: readers only consider versions whose manifest exists, so an interrupted upload is never visible. One logical PUT is therefore two S3 writes (the data, in parts above 8 MiB, then the manifest), and one GET of a known version is two S3 reads (the manifest, then the data). Finding the newest version needs an additional LIST, which the benchmark avoids by reading versions it knows; a client that always asks for "the latest" would pay it.

In N1 one encapsulation yields a root S_r, and object i of

Algorithm 1 Per-object PUT and GET (mode P1)

procedure PUT(key, M, ek)
  $v \leftarrow$ UUIDv7(); $I \leftarrow (tenant, bucket, key, v)$
  $(C, S) \leftarrow$ ML-KEM.Encaps(ek)
  $salt \xleftarrow{\$} \{0,1\}^{256}$; $p \xleftarrow{\$} \{0,1\}^{32}$
  $ctx \leftarrow$ CANON(I, mode, n, $|M|$, id(ek), H(C), ...)
  $K \leftarrow$ HKDF(S, $salt$, "pqc-s3-object-v2"$\|0\|ctx$)
  for $j \leftarrow 0$ **to** $n-1$ **do** ▷ 4 MiB chunks
    $c_j \leftarrow$ AES-GCM$_K(p\|\text{be}_{64}(j), M_j, ctx\|j\|\ell_j\|final_j)$
  end for
  upload $c_0\| \ldots \|c_{n-1}$ in 8 MiB parts to `key.pqcs/v/data.enc`
  commit: write $(ctx, C, salt, p)$ to `key.pqcs/v/manifest.json`
end procedure
procedure GET(key, dk, mode)
  $v \leftarrow$ newest version with a manifest; read and parse it
  reject unless identity, mode, key id and H(C) all match
  $S \leftarrow$ ML-KEM.Decaps(dk, C); $K \leftarrow$ HKDF(S, $salt$, ...)
  for each chunk c_j of `data.enc` **do**
    $M_j \leftarrow$ AES-GCM$_K^{-1}(\ldots)$; **reject** on failure
  end for
  reject unless exactly n chunks were read and the last is final
end procedure

the session uses

$$K_i = \text{HKDF}(S_r, salt_i, \text{"pqc-s3-session-v1"}\|0\|ctx_i),$$

with the session identifier, the object's sequence number and the bound authenticated in ctx_i. Every object still stores the session's ML-KEM ciphertext, so it can be decrypted alone. A root protects at most $B_{\max}$ objects and lives at most $T_{\max}$ seconds, whichever is reached first. In a queue-aware variant the object bound of each new session is

$$B_t = \text{clip}\big(B_{\min}, B_{\max}, \text{round}(B_{\min} + (B_{\max} - B_{\min})\hat{\rho})\big),$$

where $\hat{\rho}$ is a moving average ($\alpha = 0.2$) of the fraction of a client process's worker threads with a request in flight. Receivers cache decapsulated roots under their own caps and reject objects whose authenticated sequence number exceeds them.

Our envelope has the structure of HPKE base mode [18]: one KEM encapsulation, a key derivation bound to application context, and AEAD encryption of a sequence of records. It differs in three ways. (1) The data key is derived with a single HKDF-SHA256 call whose *info* is the object context, where HPKE uses a labelled extract-and-expand key schedule that also outputs a base nonce. (2) Chunk nonces are a random prefix followed by a counter, where HPKE computes base nonce XOR sequence number. (3) The chunk index, length and final-chunk flag are authenticated in the associated data; HPKE leaves truncation protection to the application. Mode P1 uses a KEM that [19] defines for HPKE (ML-KEM-768). That draft also defines an X25519+ML-KEM-768 hybrid under the KEM identifier that the X-Wing draft [17] requests, so P3 corresponds to it; we did not test interoperability. A deployed example of this approach is the file-encryption tool age, whose version 1.3.0 added HPKE-based hybrid ML-KEM-768 recipients [20]. It shows that a per-file post-quantum KEM is already practice; we found no performance study of it under concurrent load. We did not implement or benchmark HPKE. Since an HPKE-based envelope would perform the same KEM operation once per object, a key schedule of a few hash calls and the same AEAD, we expect the per-object costs reported here to transfer, but that expectation is untested, and a new design should use HPKE and not this format.

Table III separates the kinds of claim this paper makes. The engineering properties are supported by design and by tests, not by a proof of the protocol as a whole.

IV. Implementation and Methodology

A. Implementation

Figure 1 shows the system and Table IV its components. The client, harness and analysis are written in Python 3.12.3; the project comprises 8,006 non-blank lines of code, including 290 automated tests.

ML-KEM-768 comes from liboqs 0.16.0 through liboqs-python 0.16.0.1 [4], [47]. We build liboqs from a pinned release with only the ML-KEM parameter sets enabled and load it from an explicit installation path, so the benchmark cannot silently pick up a different library; a start-up check verifies the library version and that ML-KEM-768 is enabled, and every call checks key and ciphertext sizes. RSA-OAEP-4096 (SHA-256), X25519, HKDF and AES-256-GCM come from pyca/cryptography 50.0.1 [48]. A registry maps each mode to its key-establishment and recovery functions, and the envelope module turns the result into a streaming chunk encryptor or decryptor, so the storage code never holds a whole object. Before any timed operation every worker process runs a warm-up that loads the libraries, performs a key generation, encapsulation, decapsulation, derivation and encryption round trip, and one full round trip in the mode under test, so that one-off costs never enter a timer. An interoperability test checks ML-KEM in both directions against an independent implementation (OpenSSL through pyca), and the manifest parser rejects unknown or missing fields, wrong types, inconsistent sizes and a key-establishment ciphertext that does not match its hash.

X-Wing uses the same liboqs build for its ML-KEM component, through the library's derandomized key-generation and encapsulation functions, and the same X25519 as mode B2, so a difference between P3 and P1 or B2 reflects the construction and not a different library. The expanded decapsulation key is cached per process, which the draft permits; P1 likewise holds its expanded key.

The storage layer implements the commit protocol of Algorithm 1 on the MinIO Python SDK 7.2.20. Ciphertext records are passed to the SDK through a file-like adapter over the encryptor's output, so an object is uploaded in 8 MiB parts while it is being encrypted; on GET the stream is decrypted chunk by chunk into a sink. For benchmarking, the HTTP client uses a connection pool sized to the worker threads and performs no automatic retries, so failures are counted rather than hidden inside latency. Auxiliary functions list committed versions, verify a commit by reading it back, delete versions and collect uncommitted data left behind by interrupted uploads.

TABLE III
CLAIM TAXONOMY

Class	Claimed	Not claimed
Primitive security	Security inherited from ML-KEM-768, HKDF-SHA256 and AES-256-GCM under their stated assumptions and correct use	Any new proof or analysis of these primitives
Hybrid construction	P3 implements X-Wing and passes the draft's test vectors, so it inherits that construction's analysis	Any security property of the P2 concatenation combiner; P2 is a performance reference only
Envelope	The structure of HPKE base mode (KEM, context-bound key derivation, AEAD)	HPKE conformance, or any analysis of HPKE carrying over to this format
Engineering properties	Identity and version binding, authenticated chunk order and final-chunk flag, commit-last visibility, mode-downgrade rejection and session bounds, each exercised by tamper and stress tests	A formal model or proof of the storage protocol; resistance to side channels
Experimental findings	Measured latency, throughput, CPU, memory, metadata and queueing behaviour of the evaluated configuration	Performance on other S3 implementations, distributed deployments, native clients other than the GET-only Go reader, other regions or hardware
Out of scope	–	Availability, traffic analysis, compromised endpoints, deletion, sender authentication, rollback to an older valid version, key commitment of the AEAD, and transport security (held constant)

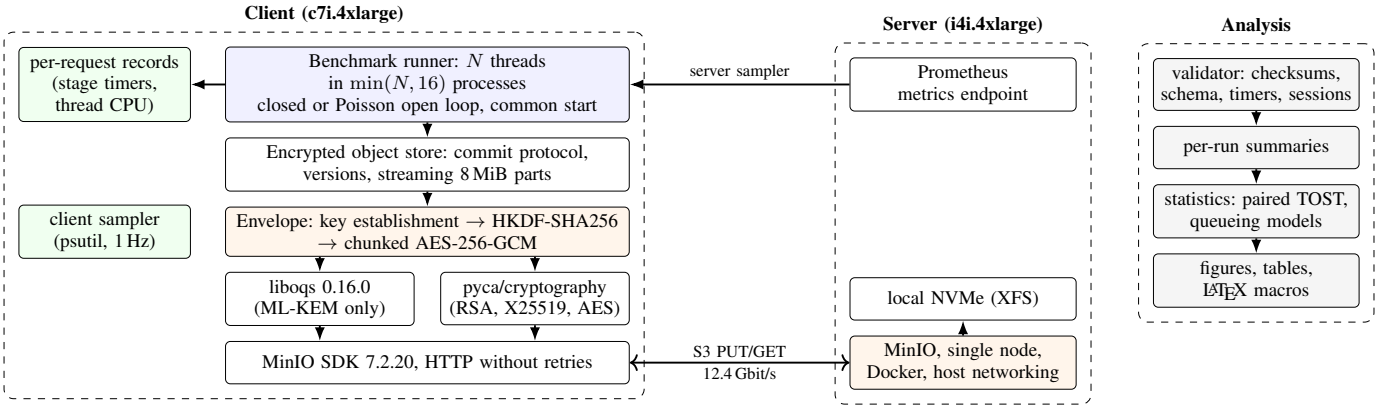


Fig. 1. System overview (testbed of the exploratory campaign; the later testbeds have the same structure). The client stack (left) issues S3 requests to MinIO (centre); every request produces a record, and client and server resources are sampled once per second. Raw data passes through a validator before any analysis (right), which generates every number in this paper.

TABLE IV
COMPONENTS OF THE IMPLEMENTATION (NON-BLANK LINES, COUNTED BY THE ANALYSIS PIPELINE)

Component	Responsibility	Lines
Cryptographic core	ML-KEM wrapper, modes, HKDF, chunked AES-GCM, manifest, sessions	1,000
Storage layer	streaming PUT/GET, commit protocol, versions, garbage collection	298
Benchmark harness	runner, payloads, samplers, records, validator, micro-benchmarks	1,176
Command-line tool	keys, settings and the `pqcstore` commands	412
Testbed automation	AWS provisioning, VM roles, MinIO image	645
Analysis pipeline	per-run summaries, statistics, paper assets	3,086
Tests	unit, interoperability, tamper, secret-hygiene and integration tests	1,389

A *run* executes one configuration (mode, operation, object size, concurrency and loop type) with freshly started worker processes; it is the unit of replication. Concurrency N is realized as exactly N worker threads spread over $\min(N, 16)$ processes, one per vCPU. Each process warms up, reports readiness, and all of them start at a common wall-clock instant, so that no worker measures while others are still starting. Each run has a 5 s warm-up phase and a 20 s measured phase; Python's garbage collector is run before and disabled during the timed window, so its pauses cannot land inside a stage timer. In closed-loop runs each thread issues requests back to back. In open-loop runs each process draws Poisson arrivals at its share of the target rate, a request waits in a queue for a free thread, and the wait is recorded separately; a request that cannot start before the end of the run is recorded as a failure, never dropped. GET runs read a set of pre-written fixture objects that are created before and deleted after the run, and every object written by a PUT run is deleted afterwards, outside the timed window.

Every request, including warm-up requests and failures, produces one record with `perf_counter` timers for the five stages of Algorithm 1 (key establishment, key derivation, AES-

GCM, manifest handling and storage), its total latency, the thread CPU time it consumed, bytes on the wire and metadata size, plus identifiers that tie it to its run, repetition, code commit and configuration. A client sampler records the CPU time and resident memory of the worker processes and the host's CPU and network counters once per second; a server sampler reads the same information from MinIO's Prometheus metrics endpoint and, in the confirmatory campaign, dirty and write-back memory, disk write rate and latency, and I/O wait from a node-exporter endpoint on the server host.

A validator checks every data set before analysis and refuses it if any check fails: checksums of all files; a complete schema with correct types for every record; that no request's stage times exceed its total; that per run the 99th percentile of unaccounted time stays below 2 % of the request time and below 1 % in aggregate; and that no N1 session exceeds its bound. The 290 automated tests cover the cryptographic core, the commit protocol, tampering (bit flips; deleted, reordered, duplicated, truncated or extended chunks; swapped, replayed or relabelled manifests; mode downgrade), that manifests contain no secret material and private-key files are protected, session bounds under concurrent stress, and end-to-end runs against a live MinIO.

A single script provisions and operates the cloud testbed: it creates both instances in one cluster placement group with a firewall that admits SSH only from the operator's address, copies the committed code, builds MinIO from source into a Docker image, formats the server's local NVMe disk at boot, runs network (iperf3, ping) and disk (fio) baselines, starts experiments in a detached session, fetches results and stops or deletes all resources. Before launch, a planner estimates the run time and cost of each experiment configuration. To bound cost, both instances stop themselves when idle, and the client stops itself shortly after an experiment completes.

As a second client (Section V-C) we wrote a deliberately small reader in Go (1.26.2): the GET path of modes B2, P1 and P3, with ML-KEM-768, X25519, HKDF, SHA-3 and AES-GCM from Go's standard library and the MinIO Go SDK. A test opens objects written by the Python client in all three modes and several sizes and rejects modified ciphertexts and a wrong expected mode, so the two clients agree on the object format. Because Go's ML-KEM takes keys as seeds, the recipient keys for this client are generated from seeds on the Python side. Its benchmark runs goroutines in one process and reports throughput, process CPU and latency; it has no stage timers.

A summarizer streams the compressed request records of each experiment once and writes one row per run. A statistics stage computes intervals, paired equivalence tests, bottleneck labels and the queueing models. A final stage generates every figure, table and number in this paper as LaTeX macros, so no result is typed by hand.

B. Testbeds and Workloads

The study has an exploratory campaign, a confirmatory campaign and a set of follow-up experiments, which we never pool.

The *exploratory campaign* (1,970 runs, 101.3 million requests) covers the full matrix with five repetitions per configuration: 1–500 workers, 1 KiB–500 MiB objects, RSA and symmetric baselines, sessions and an open loop. Its equivalence verdicts used a 95 % interval inside ±5 % and no multiplicity control; we report them as exploratory.

The *confirmatory campaign* (1,495 runs, 97.6 million requests, none of which failed, including the 185 open-loop runs, where every queued request was eventually served) was pre-registered: hypotheses, primary cells, test, margin, multiplicity correction, repetitions, stall rule, the training and held-out split for the queueing model and the session grid were written into the project's versioned specification and committed before any of its data existed. Table V maps the purpose labels used in this paper to the original batch identifiers of the artifact.

The timestamps of the pre-registration show one thing and leave others open. The commit that contains the pre-registration was pushed to a hosted repository (GitHub) on 10 October 2026 at 16:19 UTC, and the first run of the campaign started on 11 October at 11:53 UTC. The hosting service's repository activity log records each push with its time and resulting commit; the authors cannot edit it, and an export of it is part of the artifact. This establishes one thing: the registered text existed on a third party's server before the first run. It does not establish what the authors had seen locally before pushing, it is not a registration with an independent registry, and it covers a repository that was private while the experiments ran and was made public only at submission, so outsiders could not observe the pushes as they happened. One registration was pushed after a run of another experiment had begun (Section IV-C, item 4). Analyses that were not pre-specified are listed in Section IV-C and are exploratory.

Follow-up experiments (Control-Manifest, Power-PUT, Control-Binding, Allocator and Native-GET; 480 runs with the Python client and 90 with a native one) were added after the confirmatory campaign had shown a client-bound deficit whose cause we could not identify. Each was pre-registered with its own decision rule, pushed to the hosted repository before its first run, and is analysed separately; none is pooled with the confirmatory campaign. They ran on the x86 instance types of the confirmatory campaign with a new server instance.

In every testbed the client and MinIO run on two instances in one cluster placement group; MinIO is a single node in Docker with host networking on an i4i.4xlarge instance (16 vCPU, local NVMe, XFS). The exploratory campaign ran in AWS region ap-south-2 with a c7i.4xlarge client (Intel Xeon Platinum 8488C, 16 vCPU, 31 GiB): 12.4 Gbit/s between the hosts, 0.33 ms round trip, 2,044/882 MiB/s disk read/write. The confirmatory campaign ran in region ap-southeast-2, on a different account: first with the same instance types (Intel Xeon Platinum 8488C; 12.4 Gbit/s, 0.32 ms, 1,970/850 MiB/s), then with a c7g.4xlarge client (AWS Graviton3, 16 vCPU) and a new server of the same type (14.9 Gbit/s, 0.16 ms, 2,010/870 MiB/s). Because the ARM testbed has a different server instance and network path, we compare ratios between

TABLE V
EXPERIMENTS BY PURPOSE LABEL, WITH THE BATCH IDENTIFIERS USED IN THE ARTIFACT AND THE PRE-REGISTRATION. EXPLORATORY TABLES AND FIGURE TITLES GENERATED BY THE PIPELINE KEEP THE BATCH IDENTIFIERS A TO D; THE RUN COUNTS OF THE EXPLORATORY CAMPAIGN ARE IN TABLE VI.

Label	Batch	Client	Runs	Purpose
Expl-Sweep	A	x86		concurrency sweep
Expl-Size	B	x86		object size, one worker
Expl-OpenLoop	C	x86		open loop, sessions on GET
Expl-SmallPUT	D	x86		small PUT with 40 s pause
Primary-x86	E, EK	x86	300, 80	primary family; EK is the knee cell, 20 repetitions
Hybrid-Matrix	F	x86	380	B2, P1, P2, P3
Sessions	G	x86	195	$B_{\max} \times T_{\max}$
Open-Loop	H	x86	210	closed-loop $\mu(n)$ and open loop, P1
Profiling	P	x86	–	profiles only; no performance numbers
Primary-ARM	I, IK	ARM	210, 60	primary family repeated (B2, P1, P3)
OneWorker-ARM	I1	ARM	60	one-worker cells
Control-Manifest	J	x86	120	X25519 with a padded manifest
Power-PUT	K	x86	200	64 KiB PUT cells, 50 repetitions
Control-Binding	L	x86	120	ML-KEM through one foreign call
Allocator	M, MT	x86	40	diagnostic, 1 MiB PUT
Native-GET	N	x86	90	native (Go) reader, GET only

TABLE VI
EXPLORATORY CAMPAIGN: RUNS, REQUESTS AND THE RESULTS EACH BATCH SUPPLIES (BATCH IDENTIFIERS AS IN TABLE V)

Batch	Runs	Requests	Result of record for
A	850	35.4 M	concurrency sweep: GET; PUT $\geq$1 MiB; hybrid P2
B	360	1.0 M	object size 1 KiB–500 MiB; key-establishment time
C	500	46.9 M	open loop; N1 on GET; queue-aware N1
D	260	18.0 M	PUT $\leq$64 KiB incl. N1 (40 s pause)

modes across architectures, never absolute throughput.

PUT and GET are measured separately, mostly in a closed loop. Each run has a 5 s warm-up and a 20 s measured phase. The exploratory campaign measures concurrency 1–500 at 64 KiB and 1 MiB (16 MiB up to 128 workers) and object size 1 KiB–500 MiB at one worker. The confirmatory campaign measures the primary cells defined below; a hybrid matrix of 1 KiB to 128 MiB at 1 and 128 workers; sessions at $B_{\max} \in \{1, 10, 100, 1000\}$ and $T_{\max} \in \{1, 10, 60\}$ s; and, for the queueing analysis reported in the supplementary material, closed-loop throughput at 1–256 workers and Poisson arrivals at 12 rates from 1,500 to 5,600 requests/s served by 256 threads, for 1 KiB and 64 KiB PUT. The pre-registered queueing protocol (alternate rates as training and held-out points, scored by error on the stable held-out rates) is stated in full there.

The metrics are defined as follows.

- *Per-object key-establishment time*: the time a request spends in encapsulation, RSA unwrapping, Diffie–Hellman or decapsulation, from the stage timers. It is sometimes loosely called "handshake overhead"; we avoid that name because no multi-message protocol is involved.
- *Throughput*: successful requests, or MiB, per second of the measured window.
- *CPU*: CPU of the client's benchmark processes and of MinIO during the measured window, and client thread CPU time per request.
- *Memory*: peak resident memory and its growth over the run.
- *Response time* (open loop): queue wait plus service time per request.
- *Metadata*: bytes of the manifest stored with each object.

Above one worker, stage timers include time spent waiting for the Python interpreter lock, so key-establishment time and stage shares are reported at one worker only.

A run starts fresh worker processes and is the experimental unit. The millions of requests inside a run sharpen that run's own estimate; they are not independent replicates, and no interval in this paper is computed over requests. Runs are organized in randomized complete blocks: one repetition contains every mode of a cell once, in shuffled order, and modes are compared only within a repetition.

Between runs the harness waits until the server host's dirty plus write-back memory is below 32 MiB and its disk write rate below 5 MiB/s for two consecutive seconds, at most 40 s. In the exploratory experiments Expl-Sweep, Expl-Size and Expl-OpenLoop this pause had silently fallen back to 5 s, which slowed some PUT runs that followed another PUT run by 12–24 %; Expl-SmallPUT repeated the affected cells with a fixed 40 s pause. In the confirmatory campaign the measured pause had a median of 3.0 s.

C. Statistical Analysis

For a cell and two modes a and b, repetition i gives the paired log ratio $d_i = \log(x_{a,i}/x_{b,i})$ of throughput. We test equivalence with two one-sided tests (TOST) [41], [49] against the bounds $[\log 0.95, \log 1.05]$:

$$H_0^- : \ \mathrm{E}[d] \leq \log 0.95, \qquad H_0^+ : \ \mathrm{E}[d] \geq \log 1.05,$$

each with a one-sided paired t-test at $\alpha = 0.05$. The modes are declared *equivalent* only if both are rejected, which is the same as the 90 % confidence interval of the ratio lying inside $[0.95, 1.05]$; the TOST p-value is the larger of the two. We also show the 95 % interval for estimation. A cell whose 90 % interval lies wholly outside the bounds is *non-equivalent*; anything else is *inconclusive*, which is not evidence of "no difference".

The margin of five percent is our smallest effect of interest and was fixed in advance, as the margin the exploratory campaign had already used. Its practical meaning is a provisioning one: a 5 % loss of capacity raises utilization by the same

factor, so a deployment planned at or below 0.71 of peak throughput stays under the stability boundary we measure for this store (supplementary material) and needs no additional capacity. That argument uses a result of this study and was made after the fact; an operator with a tighter target needs a tighter margin, which is why we report sensitivity at $\pm 2\,\%$ and $\pm 10\,\%$ [42].

The pre-registered hypothesis is the ratio P1/B2 in eight closed-loop cells chosen before the campaign: 64 KiB GET at 128 and 500 workers; 64 KiB PUT at 32 workers (the knee of the throughput curve) and 128; 1 MiB PUT and GET at 32; and 16 MiB PUT and GET at 32. The eight p-values are adjusted with Holm's procedure [50], which controls the family-wise error rate. A limitation of this family, recognized after the campaign: only one cell, 64 KiB GET at 500 workers, has a saturated client. The 64 KiB PUT cells are limited by the server and the four cells at 1 MiB and above by the network or the server; at 64 KiB GET with 128 workers the client is the busiest resource but still has spare CPU, which a costlier mode simply uses (see the CPU accounting below). In seven of eight cells, therefore, no key-establishment mode could have changed throughput, and equivalence there confirms where the bottleneck is and little about ML-KEM. Throughput is also an insensitive endpoint in those cells; we therefore add client CPU per request as an exploratory second endpoint (Table XIII). The same family measured on the ARM client is a replication and is adjusted in the same way. All other comparisons (P3 and P2 against P1 and B2, R1 against P1, and every cell outside the family) are exploratory and reported with unadjusted intervals.

The number of repetitions follows from a power calculation. From the paired log-ratio standard deviations of the exploratory campaign, a normal-approximation calculation at $\alpha = 0.05/8$ and power 0.8 required at most 7 repetitions in every primary cell except the knee, which required about 19. We therefore ran 10 repetitions per cell and 20 at the knee, with no optional stopping.

The tests rest on a distributional assumption. The t-tests assume approximately normal mean log ratios from 10 or 20 pairs. As a check we recompute each primary verdict from a percentile-bootstrap $90\,\%$ interval of the mean log ratio (20,000 resamples).

The stall rule was fixed in advance and is blind to the mode: a closed-loop run is a *stall* if its server CPU is below 0.8 times and its throughput below 0.9 times the median of its own cell. The primary analysis uses all runs; excluding stalls is a sensitivity analysis.

The second tool of the analysis is a CPU accounting identity. Throughput is cores in use divided by CPU per request. If the client CPU is exhausted, two modes use the same number of cores, and the throughput ratio of mode a to mode b is the inverse ratio of their client CPU per request,

$$\frac{X_a}{X_b} = \frac{c_b}{c_a}.$$

This is an accounting identity, not an empirical finding and not a model with free parameters: evaluated on the runs it describes, its error measures only how equal the core counts of the two modes are. We use it as a method, in two ways. It localizes a throughput difference: if the identity holds, the difference has no source other than client CPU. And it gives an operator a procedure: measure CPU per request for the current and the candidate mode on the real client, check whether the client is saturated, and read off the expected change.

The saturation criterion differs between the two clients. For the native reader the criterion was fixed in advance: a cell is client-saturated if process CPU is at least 0.95 of the cores made available, in every mode. For the Python client no criterion was pre-registered, and we do not claim a sharp one. In the 64 KiB GET cells the post-quantum modes use nearly the same number of cores as X25519 at 500 workers and visibly more at 128 workers (Table XV); RSA-4096 uses more cores than X25519 in both. We therefore write "at or near saturation" and describe the regime by what was observed (next paragraph), not by a threshold.

A throughput deficit is defined by one rule. To say where ML-KEM lowered throughput we apply one rule to every closed-loop cell in which the Python client ran both ML-KEM and X25519: a *deficit* if the $90\,\%$ interval of the paired throughput ratio lies below 0.98, an *advantage* if it lies above 1.02. This is the rule pre-registered for the native reader; applying it to the other experiments was decided afterwards and is exploratory (see the end of this subsection).

The identity becomes a prediction only when applied to data it was not computed from. We did that once, with a ratio fixed before two follow-up experiments; it missed (Section V-C).

There were five deviations from the pre-registration. (1) The first attempt at Primary-x86 (batch E) was cut off after 134 of 300 runs when an account-level spending guard stopped both instances. Its data was never downloaded or analysed; the batch was restarted from its first run with the same configuration and seed. (2) Profiling was run twice, once under system-call tracing and once under CPU sampling, because the first CPU profiles were unusable. (3) The stall rule is applied to closed-loop runs only, since open-loop throughput is fixed by the offered rate. (4) The pre-registration of Control-Binding was pushed five minutes after Control-Manifest had started; it concerns Control-Binding and a prediction for Control-Manifest, none of whose data had been downloaded. (5) The disk baseline of the testbed of the follow-up experiments did not complete; its network baseline did.

Several analyses were not pre-specified. The pre-registration fixed the primary family, its test and the training and held-out split. It did not fix the following, which were chosen during analysis and are exploratory: applying Holm's correction to the ARM family as well; client CPU per request as a second endpoint; the CPU accounting described above, the saturation criterion for the Python client, the sweep of all cells with the deficit rule and the stage comparison of Section V-C; the single-thread timing of the native reader's key recovery; the bootstrap check; the definition of the knee as twice the light-load response time; the functional form of the load-dependent model; the order-and-time regression of Section VI; and the

TABLE VII
EXPLORATORY CAMPAIGN: KEY ESTABLISHMENT PER REQUEST (MEAN [95 % CI], ONE WORKER, OBJECTS UP TO 16 MIB, EXPL-SIZE) AND MANIFEST SIZE. MANIFESTS WERE RECORDED FOR A 1 KIB OBJECT, EXCEPT P2, RECORDED FOR A 1 MIB OBJECT; THE MANIFEST EMBEDS OBJECT-DEPENDENT FIELDS SUCH AS THE PLAINTEXT SIZE AND THE OBJECT KEY, SO THAT VALUE DIFFERS BY 6 BYTES FROM P2'S IN TABLE XXI, WHERE ALL MODES ARE RECORDED FOR 1 KIB OBJECTS.

Mode	PUT (ms)	GET (ms)	Manifest (B)
B0 plaintext	–	–	522
B1 AES only	0.001 [0.001, 0.001]	0.001 [0.001, 0.001]	576
R1 RSA-4096	0.094 [0.094, 0.094]	1.486 [1.484, 1.488]	1,276
B2 X25519	0.087 [0.086, 0.088]	0.049 [0.049, 0.049]	622
P1 ML-KEM-768	0.045 [0.045, 0.046]	0.047 [0.046, 0.047]	2,034
P2 hybrid	–	–	2,091

TABLE VIII
CONFIRMATORY CAMPAIGN: PER-OBJECT KEY-ESTABLISHMENT TIME (MS, MEAN OF FIVE RUNS, ONE WORKER, 64 KIB OBJECTS; HYBRID-MATRIX AND ONEWORKER-ARM). P2 WAS NOT MEASURED ON ARM.

	x86 (ms)		ARM (ms)	
Mode	PUT	GET	PUT	GET
B2 X25519	0.089	0.049	0.109	0.068
P1 ML-KEM-768	0.045	0.047	0.058	0.063
P2 hybrid	0.133	0.097	–	–
P3 X-Wing	0.142	0.133	0.177	0.167

layout of the session and hybrid tables.

V. RESULTS

A. Per-Object Cost

Table VII gives key-establishment time per request in the exploratory campaign, and Table VIII in the confirmatory campaign on both architectures. On the x86 client ML-KEM-768 takes 0.045 ms to encapsulate on PUT and 0.047 ms to decapsulate on GET, in agreement with the exploratory values (0.045 ms and 0.047 ms). X25519 takes 0.089 ms on PUT, where the sender also generates an ephemeral key, and 0.049 ms on GET. RSA-4096 is cheap to encrypt but takes 1.486 ms to decrypt, 32× ML-KEM decapsulation. The ARM client is slower in absolute terms (0.058 ms and 0.063 ms for ML-KEM) with the same ordering of modes. These are times through a Python binding and include its call overhead; they are not primitive benchmarks.

Finding 1. Per-object ML-KEM-768 costs 0.045–0.047 ms on x86 and 0.058–0.063 ms on ARM in this stack: less than X25519 on PUT, comparable on GET, and far less than RSA-4096 decryption.

B. Where the Cost Appears

Fig. 2 shows median latency against object size at one worker. For P1 the share of request time spent in ML-KEM falls from 1.41 % (PUT) and 2.19 % (GET) at 1 KiB to 0.012 % and 0.010 % at 500 MiB; even a 1 KiB PUT takes 3.18 ms, dominated by its two storage round trips. The only mode visibly apart is R1 on GET: its median latency is 73 % above P1 at 64 KiB and 34 % at 1 MiB.

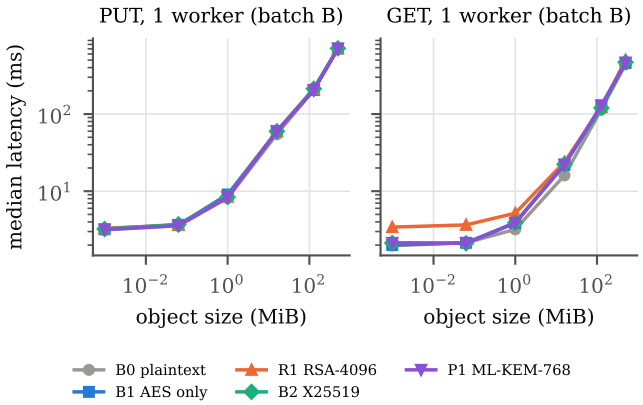


Fig. 2. Median latency against object size, one worker (Expl-Size; log scales).

Fig. 3 shows throughput from 1 to 500 workers. All modes reach the same ceilings. GETs of 1 MiB and more saturate the network at 1,477 MiB/s, 100 % of the iperf3 baseline; 1 MiB PUTs level off at about 1,310 MiB/s. Small PUTs level off at about 4,700 requests/s while the client uses at most 49 % of its CPU, the server 72 % and the network 21 % of the link. Small GETs are limited by the client CPU (93 % for P1 at 128 workers). RSA-4096 reaches that limit much earlier and delivers 0.38–0.59 of P1's throughput at 8–500 workers in the exploratory campaign and 0.42–0.46 at 128 and 500 workers in the confirmatory one.

The small-PUT ceiling is attributed by profiling the server directly. In the exploratory campaign we could only infer the cause of the small-PUT ceiling by elimination. The confirmatory campaign profiles the server directly during 1 KiB and 64 KiB PUT at 128 workers (Table IX). Three observations locate it. The device is not the limit: over 292 loaded seconds the data disk's median write latency was 0.20 ms (95th percentile 0.58 ms) with a mean queue depth of 3.0, while it wrote 283 MiB/s against an 850 MiB/s baseline; requests at that load take tens of milliseconds. The device's reported utilization was 76 %, but for an NVMe device that figure is the share of time with at least one request in flight, not a measure of saturation; latency and queue depth are the informative quantities. System-call time goes to locks and the namespace: of the time traced inside MinIO's system calls, 45 % is lock waiting (futex) and 35 % is directory and metadata operations, which are 65 % of all calls; writing the data is 5 %. MinIO does call `fdatasync`, but those calls are 0.5 % of all calls and 1.1 % of the traced time. The tracing windows also contain the deletion of objects after each run, so we report shares and not counts per upload. The CPU profile agrees: 68 % of MinIO's CPU samples are in its own code and 31 % in the kernel, where lock-spinning routines alone account for 4 % of all samples. (Across the 40 runs of the 64 KiB PUT cell at 128 workers, throughput and server CPU use move together, with correlation 0.91; that is consistent with a server-side origin of the variation but is not evidence of cause, since both rise with

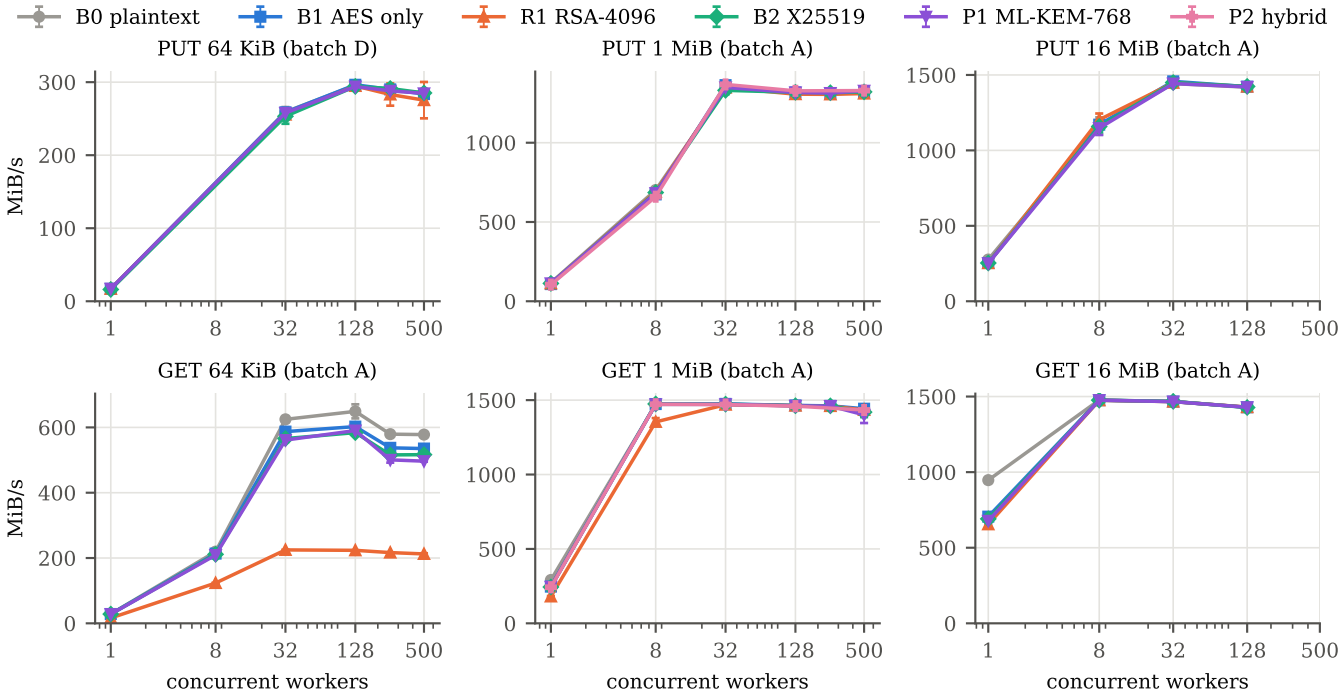


Fig. 3. Exploratory campaign: throughput (MiB/s, mean and 95 % CI over five runs) against concurrent workers. 64 KiB PUT from Expl-SmallPUT, others from Expl-Sweep.

TABLE IX
WHERE MINIO'S SYSTEM-CALL TIME GOES DURING SMALL-OBJECT PUT AT 128 WORKERS (14 TRACING WINDOWS, PROFILING). RUNS UNDER TRACING ARE SLOWED AND SUPPLY NO PERFORMANCE NUMBERS.

System-call group	Traced time	Calls
Lock waiting (futex)	44.8%	3.4%
Namespace and metadata	34.8%	64.6%
Data write	5.4%	5.1%
Read (network and files)	5.3%	11.0%
Durability (fdatasync)	1.1%	0.5%

load.) We could not resolve function names inside the MinIO binary, so we attribute the ceiling to *contention in MinIO's and the file system's metadata path on this host*, not to a named MinIO component.

Table X lists CPU and memory for representative workloads of the exploratory campaign. Per request, P1 uses between −0.045 and 0.048 ms more client CPU than X25519 at one worker, and 1.37–1.63 ms less than RSA-4096 on GET. Under contention the gap is larger: at 500 workers on 64 KiB GET, ML-KEM needs 79 µs more than X25519 on x86 and 109 µs more on ARM (Section V-C). Server CPU does not depend on the mode. Client memory is dominated by the interpreter and buffers: about 2,060 MiB at 500 workers, within 2.5 % across modes; streaming keeps the growth of a single 500 MiB PUT at most 20 MiB. Server memory growth has overlapping intervals across modes in 4 of 4 workloads. The one large value in Table X, P1 on 1 MiB PUT, is a single run in which MinIO grew its heap by 756 MiB; the other four runs of that cell grew by at most 20 MiB.

Table XII and Fig. 4 give the pre-registered result (Primary-x86, Primary-ARM). Seven of the eight cells were not client-saturated, so in them no key-establishment mode could have changed throughput; their verdicts confirm where the bottleneck is and say little about ML-KEM. With that stated: on the x86 client ML-KEM-768 is equivalent to X25519 in 6 of the 8 cells after Holm correction and inconclusive in 2; no cell is non-equivalent. The ARM replication also yields 6 equivalent and 2 inconclusive cells, though not the same ones. The informative cell is the saturated one, 64 KiB GET at 500 workers.

We first look at all cells. Applying the deficit rule of Section IV-C to all 102 closed-loop cells in which the Python client ran both modes, across every campaign, ML-KEM shows a deficit in 7. All of them are GET of objects of at most 64 KiB with at least 128 workers, and in all of them the client used at least 14.2 of its 16 cores. Cores in use mark the regime without separating it sharply: 4 other cells were at least that busy and showed no deficit. ML-KEM shows an advantage in 2 cells, both PUT at one worker in the exploratory campaign (Section V-G).

For large objects the verdict is equivalence throughout. All four cells at 1 MiB and 16 MiB are equivalent on both architectures, with ratios between 0.998 and 1.001 on x86 and between 0.992 and 1.001 on ARM. Here the network or the server sets the pace.

TABLE X
EXPLORATORY CAMPAIGN: CPU (% OF THE HOST'S 16 VCPU, MEASURED WINDOW), CLIENT CPU TIME PER REQUEST AND MEMORY (PEAK RSS AND GROWTH OVER THE RUN), MEAN OF FIVE RUNS

Workload	Mode	Client CPU %	Server CPU %	CPU ms/req	Client peak MiB	Client growth MiB	Server growth MiB
GET 64 KiB c128 (A)	B1	91	71	1.49	2,144	367	4
GET 64 KiB c128 (A)	R1	98	23	4.36	1,911	125	5
GET 64 KiB c128 (A)	B2	92	69	1.54	2,134	349	0
GET 64 KiB c128 (A)	P1	93	70	1.56	2,110	331	1
PUT 1 MiB c128 (A)	B1	34	53	3.92	2,283	265	16
PUT 1 MiB c128 (A)	R1	42	51	4.97	2,192	116	37
PUT 1 MiB c128 (A)	B2	35	52	4.06	2,298	126	33
PUT 1 MiB c128 (A)	P1	35	51	4.07	2,291	254	158
PUT 500 MiB c16 (B)	B1	25	47	675.42	2,698	174	70
PUT 500 MiB c16 (B)	R1	25	47	676.55	2,719	188	27
PUT 500 MiB c16 (B)	P1	26	47	680.14	2,729	202	18
GET 500 MiB c16 (B)	B1	23	6	1,033.11	2,011	12	90
GET 500 MiB c16 (B)	R1	23	6	1,024.33	2,025	28	144
GET 500 MiB c16 (B)	P1	22	6	997.48	2,011	33	67

The saturated cell is the one that shows a cost. At 64 KiB GET with 500 workers ML-KEM delivers 4.0 % less than X25519 on x86 (90 % CI 0.959–0.962) and 4.9 % less on ARM (0.950–0.953). The intervals are narrow, so this is a measured cost, not noise: on x86 it lies just inside the margin, so the cell is formally equivalent and at the same time significantly different from 1 (its 95 % interval excludes 1), on ARM the interval touches the bound and the verdict is inconclusive, and in neither case does it pass at ±2 %. At 128 workers the same workload shows no difference.

In small PUT the inconclusive verdicts reflect noise, not a difference. The two 64 KiB PUT cells have ratios near 1 (1.000 and 1.017 on x86) but paired standard deviations of 8–15 % in log ratio, the server-side variation described above. The knee cell passes on its own on x86 ($p = 0.041$) but not after correction ($p_{\text{Holm}} = 0.083$), and passes after correction on ARM. The cell at 128 workers is inconclusive on both: at its observed standard deviation ten repetitions give a single test a power of only 3 %, and about 50 would be needed for 80 %. Our power calculation, based on the exploratory campaign's lower variance, was too optimistic for this cell.

We followed up with adequate power. Because those two cells were under-powered, we pre-registered and ran them again with 50 repetitions each (Power-PUT, 200 runs, Table XI). Both are now equivalent: 0.994 (90 % CI 0.984–1.004) at 32 workers and 0.996 (0.979–1.013) at 128, also when the 6 stalled runs are excluded. This is a new test on new data, on another server instance; it does not change the verdicts of Table XII, which stand as the outcome of the original family.

The cost is visible in CPU even where throughput does not move. Table XIII gives the ratio of client CPU per request. Its interval excludes 1 in 3 x86 and 6 ARM cells. On 64 KiB GET ML-KEM needs more CPU than X25519; on 64 KiB PUT it needs slightly less, as its encapsulation is cheaper than X25519's ephemeral key generation and exchange.

The verdicts were checked for sensitivity to the test, the margin and stalled runs. A percentile bootstrap reproduces the unadjusted t-based verdict in 15 of 16 cells; the exception is the ARM client-bound cell, whose interval sits on the 0.95 bound and which the bootstrap places just inside it. At ±10 % all 8 x86 cells and 7 ARM cells are equivalent; at ±2 %, 5 and 4. Excluding stalled runs (1 pair on x86, 3 on ARM) and re-applying the correction changes 0 verdicts on x86 and 1 on ARM: the ARM knee cell, with one pair fewer, becomes inconclusive ($p_{\text{Holm}} = 0.076$). That cell's equivalence on ARM is therefore fragile.

TABLE XI
POWER-PUT: THE TWO 64 KIB PUT CELLS OF THE PRIMARY FAMILY REPEATED WITH 50 REPETITIONS. PAIRED THROUGHPUT RATIO ML-KEM-768/X25519, TOST AT ±5 %, HOLM-ADJUSTED OVER THE TWO CELLS; THE LAST COLUMN GIVES THE NUMBER OF PAIRS AND THE VERDICT WHEN STALLED RUNS ARE EXCLUDED.

Cell	n	Ratio	90% CI	95% CI	SD	p_{Holm}	Verdict	Without stalls
64 KiB PUT, 32	50	0.994	0.984–1.004	0.982–1.006	4 %	<0.001	Eq.	50: Eq.
64 KiB PUT, 128	50	0.996	0.979–1.013	0.976–1.017	7 %	<0.001	Eq.	44: Eq.

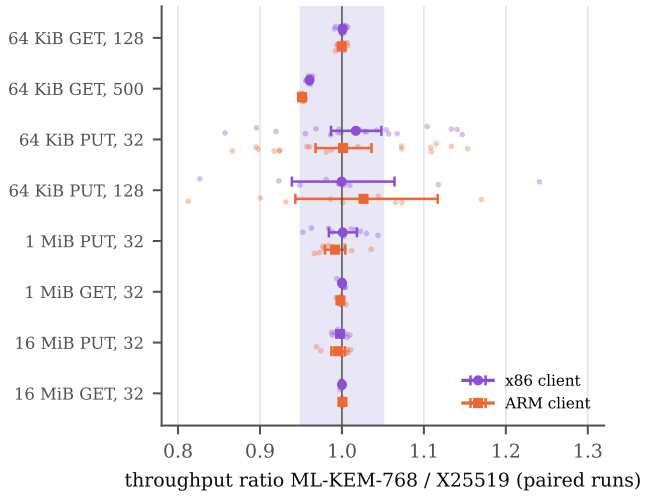


Fig. 4. Primary family. Small dots: the paired ratio of every repetition. Markers and bars: mean ratio and its 90 % interval. Shaded: the ±5 % equivalence region.

TABLE XII

PRIMARY FAMILY: PAIRED THROUGHPUT RATIO ML-KEM-768/X25519, TOST WITH BOUNDS [0.95, 1.05], $\alpha = 0.05$, HOLM-ADJUSTED OVER THE 8 CELLS OF EACH CLIENT. EQ. = EQUIVALENT, INC. = INCONCLUSIVE. THE $\pm 2\,\%$ COLUMN REPORTS WHETHER THE CELL IS ALSO EQUIVALENT AT THAT STRICTER MARGIN (UNADJUSTED).

		x86 client (Primary-x86)						ARM client (Primary-ARM)			
Cell (size, op., workers)	n	Ratio	90% CI	95% CI	p_{Holm}	Verdict	$\pm 2\%$	Ratio	90% CI	p_{Holm}	Verdict
64 KiB GET, 128	10	1.001	0.998–1.004	0.997–1.004	<0.001	Eq.	yes	1.000	0.997–1.003	<0.001	Eq.
64 KiB GET, 500	10	0.960	0.959–0.962	0.959–0.962	<0.001	Eq.	no	0.951	0.950–0.953	0.136	Inc.
64 KiB PUT, 32	20	1.017	0.987–1.048	0.980–1.055	0.083	Inc.	no	1.001	0.968–1.036	0.040	Eq.
64 KiB PUT, 128	10	1.000	0.939–1.064	0.925–1.080	0.092	Inc.	no	1.026	0.943–1.117	0.316	Inc.
1 MiB PUT, 32	10	1.001	0.984–1.018	0.980–1.022	<0.001	Eq.	yes	0.992	0.979–1.004	<0.001	Eq.
1 MiB GET, 32	10	1.000	0.998–1.002	0.998–1.002	<0.001	Eq.	yes	0.998	0.996–1.000	<0.001	Eq.
16 MiB PUT, 32	10	0.998	0.993–1.002	0.992–1.003	<0.001	Eq.	yes	0.995	0.987–1.004	<0.001	Eq.
16 MiB GET, 32	10	1.000	0.999–1.001	0.999–1.001	<0.001	Eq.	yes	1.001	1.000–1.001	<0.001	Eq.

TABLE XIII

PRIMARY CELLS: PAIRED RATIO OF CLIENT CPU PER REQUEST, ML-KEM-768/X25519, WITH 95 % INTERVAL (EXPLORATORY SECOND ENDPOINT, UNADJUSTED)

Cell	x86 client	ARM client
64 KiB GET, 128	1.018 [1.015, 1.020]	1.025 [1.021, 1.028]
64 KiB GET, 500	1.046 [1.044, 1.049]	1.061 [1.060, 1.063]
64 KiB PUT, 32	0.990 [0.983, 0.997]	0.986 [0.982, 0.990]
64 KiB PUT, 128	0.992 [0.977, 1.008]	0.999 [0.970, 1.030]
1 MiB PUT, 32	1.002 [0.996, 1.008]	0.997 [0.995, 0.998]
1 MiB GET, 32	1.006 [0.995, 1.018]	1.005 [0.999, 1.011]
16 MiB PUT, 32	1.020 [0.997, 1.043]	1.031 [1.010, 1.053]
16 MiB GET, 32	1.012 [0.994, 1.030]	1.022 [1.005, 1.040]

TABLE XIV

EXPLORATORY CAMPAIGN: P1 AGAINST X25519 BY ITS ORIGINAL INTERVAL RULE (EQUIVALENT / INCONCLUSIVE / DIFFERENT, MARGIN $\pm 5\,\%$)

Batch	Op	Sizes	Workers	Eq./Inc./Diff.	P1/B2 throughput
A	GET	1 MiB, 16 MiB	1, 8, 32, 128, 256, 500	9/1/0	0.981–1.005
A	GET	64 KiB	1, 8, 32, 128, 256, 500	6/0/0	0.961–1.011
A	PUT	1 MiB, 16 MiB	1, 8, 32, 128, 256, 500	9/1/0	0.990–1.014
B	GET	1 MiB, 16 MiB, 128 MiB, 500 MiB	1	3/1/0	0.921–1.011
B	GET	1 KiB, 64 KiB	1	2/0/0	0.991–1.000
B	PUT	1 MiB, 16 MiB, 128 MiB, 500 MiB	1	4/0/0	1.000–1.039
D	PUT	1 KiB, 64 KiB	1, 32, 128, 256, 500	5/3/0	0.989–1.060

For context, Table XIV summarizes the P1/B2 ratios of the exploratory campaign over 44 cells, judged by its original rule (95 % interval inside $\pm 5\,\%$, five runs, no multiplicity control): 38 equivalent, 6 inconclusive, 0 different. It showed the same client-bound deficit at 64 KiB GET and 500 workers (3.9 %). These counts are descriptive and are not a confirmatory result.

Finding 2. Only one of the eight pre-registered cells has a saturated client. In it the Python client is 4.0 % (x86) and 4.9 % (ARM) slower with ML-KEM than with X25519. In the other seven the modes are equivalent or, for small-object writes, inconclusive until repeated with adequate power on x86.

TABLE XV

CPU ACCOUNTING ON 64 KIB GET: THE INVERSE RATIO OF CLIENT CPU PER REQUEST (COLUMN "IMPLIED BY CPU", COMPUTED FROM THE SAME RUNS, SO NOT A PREDICTION IN THE STATISTICAL SENSE) AGAINST THE OBSERVED THROUGHPUT RATIO TO X25519 (PRIMARY-X86 AND PRIMARY-ARM)

Client	Workers	Mode	Client cores (B2 / mode)	CPU ratio	Implied by CPU	Observed	Error (pp)
x86	128	P1 ML-KEM-768	14.4 / 14.6	1.018	0.983	1.001	−1.8
x86	128	P2 hybrid	14.4 / 14.7	1.050	0.953	0.977	−2.4
x86	128	P3 X-Wing	14.4 / 14.8	1.080	0.926	0.955	−2.9
x86	128	R1 RSA-4096	14.4 / 15.6	2.597	0.385	0.420	−3.5
x86	500	P1 ML-KEM-768	14.8 / 14.9	1.046	0.956	0.960	−0.5
x86	500	P2 hybrid	14.8 / 14.9	1.080	0.926	0.930	−0.4
x86	500	P3 X-Wing	14.8 / 14.9	1.111	0.900	0.904	−0.4
x86	500	R1 RSA-4096	14.8 / 15.3	2.368	0.422	0.440	−1.7
ARM	128	P1 ML-KEM-768	13.8 / 14.1	1.025	0.976	1.000	−2.4
ARM	128	P3 X-Wing	13.8 / 14.2	1.086	0.921	0.951	−3.0
ARM	500	P1 ML-KEM-768	14.2 / 14.4	1.061	0.942	0.951	−0.9
ARM	500	P3 X-Wing	14.2 / 14.3	1.125	0.889	0.896	−0.6

C. Why the Saturated Client Is Slower

The deficit in the saturated cell needs an explanation, because on GET ML-KEM's decapsulation is *cheaper* than X25519's exchange (Table VIII).

Table XV applies the identity of Section IV-C to 64 KiB GET. At 500 workers, where the post-quantum modes and X25519 use the same number of cores to within 0.1, the inverse CPU ratio matches the observed throughput ratio of ML-KEM and of both hybrids to within 0.9 percentage points on both architectures, and that of RSA-4096, which needs more than twice the CPU, to within 1.7. At 128 workers the costlier mode uses more cores, throughput holds, and the identity, which assumes equal cores, overstates the loss by 1.8–3.0 points for the post-quantum modes and 3.5 for RSA-4096. In the saturated cell the deficit of Section V-B is therefore a difference in client CPU: ML-KEM needs 79 µs more per request than X25519 on x86 (of 1.71 ms) and 109 µs more on ARM.

The stage timers show where the extra CPU is not. Table XVI compares the stage timers of ML-KEM and X25519 at one worker, where they are not distorted by interpreter-lock waiting. On GET, key establishment is *faster* with ML-KEM on both architectures. The stage that is slower by the most is manifest handling, which parses the manifest, decodes the ciphertext and verifies its hash (+6.4 µs on x86, +5.5 µs on ARM; +16.2 µs and +16.4 µs on PUT, which encodes and hashes it). Key derivation is also slower (+4.6 µs and +4.6 µs

TABLE XVI
ML-KEM-768 MINUS X25519 AT ONE WORKER, 64 KiB OBJECTS (HYBRID-MATRIX AND ONEWORKER-ARM, FIVE RUNS): DIFFERENCE IN MEAN STAGE TIME, AND IN CLIENT CPU PER REQUEST WITH ITS 95 % INTERVAL

	Stage time, P1 − B2 (µs)				(µs, 95 % CI)
Client, op.	Key est.	Key deriv.	AES-GCM	Manifest	CPU per request
x86 PUT	−44.0	+4.0	+0.1	+16.2	−41 [−92, +10]
x86 GET	−2.1	+4.6	+0.2	+6.4	+21 [−41, +83]
ARM PUT	−50.6	+4.8	+0.1	+16.4	−22 [−42, −2]
ARM GET	−4.7	+4.6	+0.2	+5.5	+6 [−23, +34]

on GET) although its inputs have the same length in both modes (Section V-G). The cipher is unaffected.

The stage timers therefore rule out decapsulation and leave two candidates: the handling of a manifest 1,412 bytes larger, and the way the client reaches ML-KEM. We tested each with a pre-registered control on 64 KiB GET at 500 workers (Table XVII; decision rules fixed before the runs).

The first control, Control-Manifest, tests manifest size. Mode C2 is X25519 whose stored ciphertext is zero-padded to 1,088 bytes, so its manifest is byte-for-byte as long as ML-KEM's. Padding alone costs 0.7 % of throughput (ratio 0.993, 90 % CI 0.991–0.995). ML-KEM remains 3.3 % slower than the padded control (0.967, 0.965–0.969). By the pre-registered rule, manifest size *does not explain* the deficit: it accounts for 17 % of it (90 % CI 14–21 %). At one worker the padded control shows the same slower manifest stage as ML-KEM (+6.0 µs against +5.6 µs), so that stage difference is purely one of size, and it is small.

The second control, Control-Binding, tests the library binding. Our client reaches liboqs through liboqs-python, which creates a key-encapsulation object for every operation and makes four foreign calls per decapsulation (allocate, decapsulate, cleanse, free), each releasing the interpreter lock; X25519, through pyca/cryptography, is a single call. Modes H1 and G1 run the same liboqs ML-KEM-768 and write the same manifest as P1, but make one foreign call per operation on a handle created once per process; H1 releases the interpreter lock around it and G1 holds it. With one call, throughput is equivalent to X25519's within ±2 %: H1 reaches 0.999 (0.997–1.002) and G1 1.002 (0.997–1.006), while P1 in the same experiment is at 0.963 (0.960–0.967). By the pre-registered rule the binding *explains* the deficit; it removes 99 % of it (90 % CI 92–105 %). Whether the single call releases the lock makes no measurable difference (G1/H1 1.002, 0.999–1.006), so the cost lies in the per-operation object handling and the number of foreign calls, not in the locking policy as such. The one-worker timers agree: a decapsulation takes 0.047 ms through the binding and 0.028 ms as a single call, against 0.050 ms for X25519.

The controls establish the following. On the x86 Python client, where both controls ran, the deficit in the saturated cell is produced by how the binding invokes the library: 17 % of it comes from the larger manifest, and with one foreign call per operation ML-KEM is equivalent to X25519 within ±2 %. The two controls are independent and their effects need not add up: the one-call modes carry the larger manifest and still match X25519. The deficit itself was observed in three independent experiments on two server instances (0.960, 0.960 and 0.963). The controls were not run on ARM (Section VI).

The one advance prediction missed. Before Control-Manifest and Control-Binding we fixed a predicted ratio of 0.956 from the CPU ratio measured in Primary-x86. The observed ratios differed from it by 0.4 and 0.7 points, and the prediction lay outside both intervals. The identity holds within an experiment; it did not carry a ratio exactly from one experiment to the next.

Finding 3. On the x86 Python client, the 4.0 % deficit in the saturated cell is produced by the binding, which makes four foreign calls per decapsulation: with one call ML-KEM is equivalent to X25519 within ±2 % (0.999 of its throughput). The larger manifest accounts for 17 % of the deficit.

If the Python client's deficit is a binding artifact, does a compiled client show none? We wrote a reader in Go that implements the GET path of modes B2, P1 and P3 and opens objects written by the Python client; it does the same per-object work (two S3 reads, manifest parsing, ciphertext-hash check, key recovery, HKDF, AES-GCM per chunk, a hash of the plaintext), with ML-KEM-768 and X25519 from Go's standard library. A compiled client needs so little CPU that 16 cores cannot be saturated by this store, so the pre-registered primary cell restricts it to one core with 128 workers, which saturates it by construction (Native-GET, 90 runs, Table XVIII).

The result for the native reader is as follows. In the saturated cell ML-KEM reaches 0.905 of X25519's throughput (90 % CI 0.900–0.910) and X-Wing 0.826 (0.821–0.831): by the pre-registered rule, a *deficit* for both. Their CPU per request is higher by the inverse factors (1.105 and 1.211); here the identity of Section IV-C holds by construction, since every mode ran on exactly one core, and this is not a validation of it. On one core the native reader spends 343 µs of process CPU per request with X25519.

Part of the native reader's extra CPU can be located. With liboqs called directly, ML-KEM decapsulation was *cheaper* than X25519 (0.028 ms against 0.050 ms). In Go's standard library it is dearer: timed alone on the same machine, decapsulation takes 50.6 µs, an X25519 exchange 33.4 µs and an X-Wing decapsulation 85.1 µs (an exploratory single-thread timing, not part of the pre-registered experiment). For ML-KEM, key recovery thus accounts for about half of the extra CPU per request (17 of 36 µs); for X-Wing, for about three quarters (52 of 72 µs). The remainder was not measured separately in this client.

With 16 cores the reader does not meet the saturation criterion (9.75 and 11.50 cores in use). At 500 workers the modes are within 1.1 % of each other. At 128 workers ML-KEM is 5.7 % and X-Wing 7.7 % slower, with wider intervals (Section V-G).

TABLE XVII
CONTROLS ON 64 KIB GET (CONTROL-MANIFEST AND CONTROL-BINDING, TEN REPETITIONS, X86 PYTHON CLIENT): PAIRED RATIOS WITH 90 % INTERVALS. "PADDED MANIFEST" = X25519 WITH A MANIFEST AS LARGE AS ML-KEM'S; "BINDING" = ML-KEM-768 THROUGH LIBOQS-PYTHON (MODE P1); "ONE CALL" = THE SAME ML-KEM-768 THROUGH A SINGLE FOREIGN CALL PER OPERATION.

Experiment, workers	Comparison	Throughput ratio	CPU-per-request ratio
Control-Manifest, 500	X25519, padded manifest / X25519	0.993 [0.991, 0.995]	1.008 [1.007, 1.009]
Control-Manifest, 500	ML-KEM-768 (binding) / X25519, padded manifest	0.967 [0.965, 0.969]	1.038 [1.036, 1.040]
Control-Manifest, 500	ML-KEM-768 (binding) / X25519	0.960 [0.958, 0.962]	1.047 [1.044, 1.049]
Control-Manifest, 128	X25519, padded manifest / X25519	0.995 [0.992, 0.998]	1.005 [1.003, 1.008]
Control-Manifest, 128	ML-KEM-768 (binding) / X25519, padded manifest	1.004 [1.002, 1.006]	1.012 [1.010, 1.013]
Control-Manifest, 128	ML-KEM-768 (binding) / X25519	0.999 [0.995, 1.002]	1.017 [1.014, 1.021]
Control-Binding, 500	ML-KEM-768 (binding) / X25519	0.963 [0.960, 0.967]	1.043 [1.040, 1.046]
Control-Binding, 500	ML-KEM-768, one call, lock released / X25519	0.999 [0.997, 1.002]	1.007 [1.004, 1.009]
Control-Binding, 500	ML-KEM-768, one call, lock held / X25519	1.002 [0.997, 1.006]	0.998 [0.995, 1.002]
Control-Binding, 500	ML-KEM-768, one call, lock released / ML-KEM-768 (binding)	1.038 [1.033, 1.042]	0.965 [0.963, 0.968]
Control-Binding, 500	ML-KEM-768, one call, lock held / ML-KEM-768, one call, lock released	1.002 [0.999, 1.006]	0.991 [0.988, 0.995]
Control-Binding, 128	ML-KEM-768 (binding) / X25519	1.005 [1.001, 1.008]	1.016 [1.013, 1.018]
Control-Binding, 128	ML-KEM-768, one call, lock released / X25519	1.000 [0.997, 1.003]	1.007 [1.004, 1.010]
Control-Binding, 128	ML-KEM-768, one call, lock held / X25519	1.002 [0.995, 1.008]	0.999 [0.994, 1.004]
Control-Binding, 128	ML-KEM-768, one call, lock released / ML-KEM-768 (binding)	0.995 [0.990, 1.000]	0.992 [0.987, 0.996]
Control-Binding, 128	ML-KEM-768, one call, lock held / ML-KEM-768, one call, lock released	1.002 [0.996, 1.008]	0.992 [0.986, 0.998]

TABLE XVIII
NATIVE GO READER, 64 KIB GET (NATIVE-GET, TEN REPETITIONS): X25519 IN ABSOLUTE TERMS (THROUGHPUT, PROCESS CPU IN CORES, CPU PER REQUEST) AND PAIRED RATIOS TO IT WITH 90 % INTERVALS. ONLY THE ONE-CORE CELL HAS A SATURATED CLIENT.

	X25519			ML-KEM-768 / X25519		X-Wing / X25519	
Cell	req/s	Cores	CPU (µs)	Throughput	CPU/request	Throughput	CPU/request
1 core, 128	2,920	1.00	343	0.905 [0.900, 0.910]	1.105 [1.099, 1.111]	0.826 [0.821, 0.831]	1.211 [1.203, 1.218]
16 cores, 128	10,300	9.75	947	0.943 [0.929, 0.958]	1.072 [1.051, 1.093]	0.923 [0.906, 0.940]	1.117 [1.090, 1.144]
16 cores, 500	14,580	11.50	789	0.996 [0.994, 0.999]	1.091 [1.054, 1.129]	0.989 [0.986, 0.992]	1.157 [1.123, 1.191]

Finding 4. A native reader saturated on one core is 9.5 % slower with ML-KEM and 17.4 % with X-Wing. Across the clients measured, the saturated-client cost of ML-KEM ranged from equivalence within ±2 % (liboqs through one call) to 9.5 % (Go's standard library).

D. Hybrid Key Establishment

We first give the cost of the hybrids per object. X-Wing takes 0.142 ms on PUT and 0.133 ms on GET on x86 (Table VIII), 3.1 and 2.8 times ML-KEM alone. The concatenation hybrid P2 takes 0.133 ms and 0.097 ms; on GET that is close to the sum of its two components (0.095 ms), whereas X-Wing takes more. The remainder is the combiner hash and our Python-level handling of the seed-derived keys, which we did not profile separately. Even so, X-Wing is at most 5.8 % of a one-worker 64 KiB request on x86 and 8.2 % on ARM, and 0.10 % of a 128 MiB PUT.

Table XIX repeats the primary family for the hybrids (exploratory, unadjusted). X-Wing is equivalent to X25519 in 6 of 8 cells on x86 and 4 on ARM. In the client-bound cell (64 KiB GET, 500 workers) it is non-equivalent on both: 9.6 % slower than X25519 on x86 and 10.4 % on ARM, and 5.8 % and 5.9 % slower than ML-KEM alone. At 128 workers the same workload already costs 4.5–4.9 %. From 1 MiB upward X-Wing is equivalent to both on both architectures. P2 is equivalent to P1 in all 8 cells, with a lowest ratio of 0.969.

Table XX extends the comparison to 18 workloads from 1 KiB to 128 MiB. X-Wing's throughput ratio to X25519 ranges from 0.91 to 1.02 and lies within ±5 % in 15 of them; the lowest values are again small GETs at 128 workers. Its client CPU per request is up to 21 % above X25519. Tail latency (p99) follows throughput and shows no separate hybrid penalty. Client peak memory differs from X25519 by a median of 4 MiB (0.4 %) and at most 5 % across modes and workloads. One entry of this table, 1 MiB PUT with one worker, is not a cryptographic effect (Section V-G).

TABLE XIX
HYBRID MODES IN THE PRIMARY CELLS: PAIRED THROUGHPUT RATIO AND UNADJUSTED TOST VERDICT AT ±5 % (EXPLORATORY). P3 = X-WING, P2 = CONCATENATION HYBRID.

	x86 client			ARM client	
Cell	P3/B2	P3/P1	P2/P1	P3/B2	P3/P1
64 KiB GET, 128	0.955 (Eq.)	0.955 (Eq.)	0.976 (Eq.)	0.951 (Inc.)	0.952 (Inc.)
64 KiB GET, 500	0.904 (Non-eq.)	0.942 (Non-eq.)	0.969 (Eq.)	0.896 (Non-eq.)	0.941 (Non-eq.)
64 KiB PUT, 32	1.023 (Eq.)	1.006 (Eq.)	0.994 (Eq.)	0.974 (Inc.)	0.973 (Inc.)
64 KiB PUT, 128	0.985 (Inc.)	0.986 (Inc.)	1.004 (Eq.)	1.019 (Inc.)	0.993 (Eq.)
1 MiB PUT, 32	1.009 (Eq.)	1.008 (Eq.)	1.005 (Eq.)	0.981 (Eq.)	0.989 (Eq.)
1 MiB GET, 32	0.998 (Eq.)	0.998 (Eq.)	0.999 (Eq.)	0.998 (Eq.)	1.000 (Eq.)
16 MiB PUT, 32	1.000 (Eq.)	1.003 (Eq.)	1.004 (Eq.)	0.989 (Eq.)	0.994 (Eq.)
16 MiB GET, 32	1.000 (Eq.)	0.999 (Eq.)	1.000 (Eq.)	1.000 (Eq.)	0.999 (Eq.)

E. Storage Overhead

The lasting cost of post-quantum key establishment is storage. Table XXI converts the per-object manifest into fleet totals. An ML-KEM manifest is 2,034 bytes, 1,412 more than X25519; X-Wing adds another 40. Two parts of that difference must be kept apart. The *intrinsic* cost of ML-KEM is its

TABLE XX
HYBRID-MATRIX (64 KiB AT 128 WORKERS FROM PRIMARY-X86): X25519 IN ABSOLUTE TERMS AND PAIRED RATIOS OF ML-KEM-768 (P1), THE CONCATENATION HYBRID (P2) AND X-WING (P3) TO IT. * = THE 95 % INTERVAL OF THE RATIO EXCLUDES 1 (UNADJUSTED, FIVE RUNS; TEN IN PRIMARY-X86). OF THE 162 RATIOS, 66 ARE MARKED; 50 REMAIN AT A FALSE-DISCOVERY RATE OF 5 % (BENJAMINI–HOCHBERG), SO INDIVIDUAL MARKS SHOULD NOT BE OVER-READ.

		X25519 (B2)		Throughput ratio			p99 latency ratio			Client CPU/request ratio		
Workload	c	req/s	MiB/s	P1	P2	P3	P1	P2	P3	P1	P2	P3
1 KiB PUT	1	298	0.3	1.00	0.99	0.97*	1.01	1.03	1.04*	0.99*	1.07*	1.07*
1 KiB GET	1	457	0.4	0.99	0.98	1.02*	1.01	1.03*	1.02*	1.01*	1.05*	1.08*
1 KiB PUT	128	5,008	4.9	1.04	1.07	1.01	0.86	0.82	0.92	1.01	1.10*	1.10*
1 KiB GET	128	11,028	10.8	0.97*	0.94*	0.91*	0.99	1.03*	1.07*	1.04*	1.08*	1.12*
64 KiB PUT	1	250	15.6	1.01	0.99	0.99	1.01	1.03	1.03*	0.97	1.03*	1.04*
64 KiB GET	1	451	28.2	1.03	1.02	0.95*	0.95	1.01	1.04*	1.02	1.04	1.13*
64 KiB PUT	128	4,569	286	1.00	1.00	0.99	0.98	0.91	0.95	0.99	1.09*	1.09*
64 KiB GET	128	9,415	588	1.00	0.98*	0.96*	0.98*	1.01*	1.04*	1.02*	1.05*	1.08*
1 MiB PUT	1	115	115	1.00	0.90*	0.94*	1.00	1.10*	1.09*	1.00	1.33*	1.21*
1 MiB GET	1	247	247	1.00	0.98*	0.97*	1.00	1.03*	1.02	1.00	1.02*	1.04*
1 MiB PUT	128	1,321	1,321	1.00	0.99	0.99*	1.00	1.01	1.02	1.01	1.27*	1.07*
1 MiB GET	128	1,464	1,464	1.00	1.00	1.00	1.00	0.98	0.99	1.01*	1.01*	1.04*
16 MiB PUT	1	16	251	0.99	0.99	0.99*	1.01	1.00	1.01	1.01	1.01	1.02
16 MiB GET	1	43	685	1.02	1.02	0.97*	0.94	0.98	1.00	0.98	0.98	1.06*
16 MiB PUT	128	89	1,421	1.00	1.00	0.99*	0.95	0.97	0.90*	1.05*	1.08*	1.06*
16 MiB GET	128	89	1,430	1.00	1.00	1.00	1.03	1.01	1.00	1.03*	1.02*	1.01
128 MiB PUT	1	5	588	1.04*	1.03	0.99	0.98	0.98	1.01	0.90*	0.93	1.03
128 MiB GET	1	8	1,017	0.98	0.98	1.01	1.00	1.02	0.98	1.03	1.02	0.99

ciphertext, 1,056 bytes more than an X25519 public key. The remaining 356 bytes are our encoding: the manifest is JSON and carries the ciphertext in base64 (1,452 characters), plus 4 bytes for a longer algorithm name. For one billion objects the difference is 1.41 TB (1.28 TiB) in this encoding and 1.06 TB with a binary one. Relative to the payload it is negligible from 1 MiB upward and larger than the payload itself at 1 KiB. These are logical bytes: replication or erasure coding multiplies them, and because the manifest is a separate S3 object in our design, it also doubles the number of stored objects and write requests, which matters under per-request billing and for inventory scans. Carrying the manifest in the object's own metadata, as the Amazon S3 Encryption Client does by default [2], would avoid the second object. Amazon S3 limits user-defined metadata to 2 KB per object [51]: our JSON manifests for ML-KEM and both hybrids (2,034–2,085 bytes) are at or above that limit, whereas a binary manifest (a 1,088-byte ciphertext and a few hundred bytes of context) would fit, provided the store accepts binary metadata values or the client pays the base64 expansion once more in the header. Our storage results therefore describe a two-object, JSON-manifest design, which we chose so that an interrupted upload is never visible and a manifest can be read without the data; they overstate both the bytes and the request count of a metadata-based design.

Finding 5. Against X25519, ML-KEM-768 adds 1,056 bytes of ciphertext per object, and 1,412 in our JSON encoding: irrelevant for large objects, dominant for kilobyte-sized ones, and 1.06–1.41 TB per billion objects before redundancy.

F. Bounded Sessions

A session changes what a compromise exposes. In P1 every object has its own shared secret, and an attacker who obtains one learns one object. In N1 the unit of compromise is the session root: whoever reads a root from process memory, a crash dump, a swap file or a log can derive the data keys of every object it protected, at most $B_{\max}$ objects written within $T_{\max}$ seconds. A cache shared between tenants would extend that exposure across tenants, so roots must be scoped to one recipient key, as ours are. This exposure is distinct from, and much smaller than, compromise of the recipient's long-term decapsulation key, which exposes all objects in every mode. The theoretical $1/B_{\max}$ operations per object holds only when sessions fill before the time limit, so we report what was observed.

TABLE XXI
MANIFEST SIZE PER OBJECT AND ITS SCALING, FOR THIS STUDY'S JSON/BASE64 MANIFEST. EXTRA BYTES ARE RELATIVE TO X25519; 1,056 OF ML-KEM'S 1,412 ARE CIPHERTEXT, THE REST IS ENCODING.

			Extra storage at object count			Share of payload		
Mode	Manifest (B)	Extra (B)	10^6	10^8	10^9	1 KiB	64 KiB	1 MiB
B2 X25519	622	–	–	–	–	–	–	–
P1 ML-KEM-768	2,034	1,412	1.4 GB	141.2 GB	1.41 TB	138%	2.15%	0.13%
P2 hybrid	2,085	1,463	1.5 GB	146.3 GB	1.46 TB	143%	2.23%	0.14%
P3 X-Wing	2,074	1,452	1.5 GB	145.2 GB	1.45 TB	142%	2.22%	0.14%

Table XXII gives the result for 1 KiB PUT, the workload where key establishment is the largest share of a request.

Operations per object follow $1/B_{\max}$ when sessions fill (0.0011 at $B_{\max} = 1000$, $T_{\max} \geq 10$ s). When the time limit binds first they do not: at $T_{\max} = 1$ s a root protected a median of 248 objects under closed-loop load (0.0038 operations per object) and 94 under light open-loop load, where even $B_{\max} = 100$ was not reached (median 94). The time limit, not the object limit, therefore bounds exposure for lightly loaded clients.

Client CPU per request falls by 2.1–3.7 % for $B_{\max} \geq 10$, and nearly all of that is reached at $B_{\max} = 10$. A session of

TABLE XXII
BOUNDED SESSIONS (N1) AGAINST PER-OBJECT ML-KEM (P1), 1 KiB PUT, SESSIONS. CPU, THROUGHPUT AND P99 ARE PAIRED RATIOS TO P1 (* = 95 % INTERVAL EXCLUDES 1; FIVE RUNS). OBJECTS AND BYTES PER ROOT ARE WHAT ONE LEAKED ROOT WOULD EXPOSE.

Load	$B_{\max}$	$T_{\max}$ (s)	KEM/object	CPU	Throughput	p99	Objects/root (med/p95/max)	Max KiB/root
closed, 32 workers	1	1	1.0000	1.019*	1.021	0.99	1/1/1	1
closed, 32 workers	1	10	1.0000	1.016	1.021	0.95	1/1/1	1
closed, 32 workers	1	60	1.0000	1.017*	1.020	0.97	1/1/1	1
closed, 32 workers	10	1	0.1001	0.979*	1.062	0.92	10/10/10	10
closed, 32 workers	10	10	0.1001	0.972*	1.016	0.97	10/10/10	10
closed, 32 workers	10	60	0.1001	0.972*	1.010	0.98	10/10/10	10
closed, 32 workers	100	1	0.0101	0.971*	1.033	0.96	100/100/100	100
closed, 32 workers	100	10	0.0101	0.963*	0.966	1.05	100/100/100	100
closed, 32 workers	100	60	0.0101	0.970*	1.043	0.95	100/100/100	100
closed, 32 workers	1,000	1	0.0038	0.969*	1.015	0.98	248/276/310	310
closed, 32 workers	1,000	10	0.0011	0.970*	1.028	0.98	1000/1000/1000	1,000
closed, 32 workers	1,000	60	0.0011	0.970*	1.025	0.97	1000/1000/1000	1,000
open, 1,500/s	100	1	0.0110	0.972*	1.003	0.98	94/100/100	100
open, 1,500/s	100	10	0.0102	0.973*	0.999	1.00	100/100/100	100
open, 1,500/s	100	60	0.0102	0.973*	1.001	1.01	100/100/100	100
open, 1,500/s	1,000	1	0.0107	0.972*	1.000	0.99	94/110/125	125
open, 1,500/s	1,000	10	0.0013	0.971*	1.003	0.97	916/980/1000	1,000
open, 1,500/s	1,000	60	0.0013	0.971*	0.993	0.98	1000/1000/1000	1,000

one object costs 1.7 % *more* CPU than P1, the bookkeeping overhead of the mechanism.

Throughput and tail latency do not change: in none of the 9 configurations with $B_{\max} \geq 10$ does the throughput interval exclude 1 (0.966–1.062), because this workload is limited by the server. On a client-bound workload, 64 KiB GET at 32 workers, a receiver that caches roots gains 3.3 % throughput (95 % CI 3.1–3.5 %) at $B_{\max} = 100$. This is an upper bound: the run reads 32 objects repeatedly, so every root stays in the receiver's cache and almost no decapsulation remains; a workload with a large working set would gain less. The exploratory campaign agrees: N1 saved 30–66 µs of CPU per request at one worker and raised throughput only for small GETs at 32–128 workers (1.4–5.1 %). Its queue-aware variant lowered the bound under light load as designed, re-keying 5.5× as often as a fixed bound of 100 at 1,500 requests/s with no measurable change in tail latency; we did not show that it is better than a smaller fixed bound.

The measurements support a conservative rule. Per-object ML-KEM is the default. Amortization is enabled only for a workload in which profiling shows the client CPU to be the bottleneck, and then with the smallest bound that achieves the gain: in our data $B_{\max} = 10$ already removes most of the CPU cost, so a larger bound buys exposure without benefit. $T_{\max}$ should be set from the tolerated exposure of an idle client, since it is the limit that binds under light load. The same reasoning applies to the choice between ML-KEM and a hybrid: where the client is not the bottleneck the hybrid costs only 40 bytes per object, and where it is, the cost is the throughput difference reported in Section V-D.

Finding 6. Sessions trade compromise scope for a 2.1–3.7 % saving in client CPU. That saving becomes throughput only on client-bound workloads; a bound of 10 objects captures most of it.

G. Open Anomalies

Five observations are not explained by anything above. We list them together so that they are not read past.

(A1) Key derivation on GET at one worker is slower with ML-KEM. At one worker the key-derivation stage is +4.6 µs (x86) and +4.6 µs (ARM) slower on GET with ML-KEM than with X25519, although its inputs have the same length in both modes. The padded-manifest control does not show it (+4.3 µs for ML-KEM over the padded control), so it is not an effect of manifest size. We have no explanation.

(A2) The native reader shows a deficit without saturation. In Native-GET with 16 cores and 128 workers ML-KEM is 5.7 % and X-Wing 7.7 % slower than X25519, in a cell that does not meet the pre-registered saturation criterion (9.75 cores in use). Two cautions apply: the 16 virtual CPUs are 8 physical cores, and the reader's CPU per request is two to three times higher with 16 cores than with one, which suggests runtime overhead that makes "cores in use" a soft measure of saturation here. This result is the one exception to the statement that ML-KEM lowered throughput only at or near client saturation.

(A3) X-Wing is below X25519 in a Python cell we call unsaturated. At 64 KiB GET with 128 workers X-Wing is 4.5 % (x86) and 4.9 % (ARM) slower than X25519 (Table XIX), although by our criterion that cell is not saturated. The client is the busiest resource there, so this may be near-saturation; we did not test it.

(A4) In some exploratory cells ML-KEM was faster than X25519 at one worker. By the deficit rule, ML-KEM has an advantage in 2 cells of the exploratory campaign, both PUT at one worker (ratios 1.04 and 1.06, five runs each). Key establishment is far too small a share of those requests to produce a difference of that size, and the corresponding cells of Hybrid-Matrix do not show an advantage by the same rule. We have no explanation; an allocator effect of the kind described next is a candidate that we did not test.

TABLE XXIII
1 MiB PUT, ONE WORKER: AES-GCM STAGE TIME AND THROUGHPUT WITH THE DEFAULT ALLOCATOR AND WITH GLIBC'S MMAP THRESHOLD FIXED AT 256 MiB (ALLOCATOR, FIVE RUNS EACH)

Mode	Default allocator AES-GCM (ms)	Default allocator req/s	Fixed threshold AES-GCM (ms)	Fixed threshold req/s
B2 X25519	0.12	115	0.29	109
P1 ML-KEM-768	0.12	116	0.29	109
P2 hybrid	0.38	104	0.29	108
P3 X-Wing	0.28	108	0.32	105

(A5) An allocator effect appears at 1 MiB. In Hybrid-Matrix, at 1 MiB PUT with one worker X-Wing is 6 % and P2 10 % slower than X25519 although key establishment is 1.5 % of the request; the difference sits in the AES-GCM stage timer, which all modes share. A diagnostic pair of experiments (Allocator, 40 runs, Table XXIII) points to the memory allocator. With default settings the AES-GCM stage takes between 0.12 and 0.38 ms depending on the mode, a factor of 3.2. With glibc's dynamic mmap threshold disabled it takes 0.29–0.32 ms in every mode, a factor of 1.1, and the throughput spread between modes falls from 10 % to 3 %. The stage time is thus governed by where the 1 MiB chunk buffer is allocated, which depends on a mode's earlier allocations, and not by the cipher or the key establishment. A small residual for X-Wing remains. The residual for X-Wing is not explained, and the diagnostic was exploratory.

VI. THREATS TO VALIDITY

We first examine run-to-run variation. Fig. 4 shows every paired ratio of the primary family. Six cells are tight: the coefficient of variation of throughput across runs is at most 2.2 %. The two 64 KiB PUT cells vary by 4–9 % per mode. That variation originates at the server (Section V-B) and affects all modes alike, which is why it widens intervals without shifting ratios.

The pre-registered stall rule flags 8 of 1,165 x86 runs and 3 of 330 ARM runs, all of them PUTs of at most 64 KiB. Excluding them leaves the 8 x86 verdicts unchanged and turns one ARM verdict, the knee cell, from equivalent to inconclusive (Section V-B).

Order and time effects were checked as follows. Modes are shuffled within each repetition. As a check, for each primary cell we regress log throughput on the mode, repetition fixed effects, elapsed time in the experiment, position in the block and whether the previous run was a PUT. The adjusted P1/B2 ratio differs from the paired ratio by at most 1.1 % in any cell on either architecture.

The ARM replication reproduces the pattern: equivalence at 1 MiB and above, a deficit of 4.0 % (x86) and 4.9 % (ARM) in the client-bound cell, and noise in small PUT. Because that testbed also has another server instance, it tests the robustness of the ratios and not ARM against x86 in absolute terms.

Threats to internal validity are the following. Stage timers above one worker include interpreter-lock waiting, so key-establishment time is reported at one worker only. During the x86 chain of the confirmatory campaign, completed experiments were downloaded from the client while parts of Hybrid-Matrix, Sessions and Open-Loop were running; the transfer used client CPU and network and may have added noise to those runs. Mode order is randomized within repetitions, so it cannot favour a mode, and no transfer overlapped Primary-x86 or Primary-ARM. Modes differ in manifest size as well as in key establishment; Control-Manifest separates the two for the Python client only (Section V-C). Seven of the eight primary cells could not have shown an effect of the mode on throughput. The exploratory campaign's pause failure (Section IV) added noise to small PUT cells; its results for those cells come from Expl-SmallPUT. The 1 MiB one-worker slowdown of the hybrids is attributed to the allocator by an exploratory diagnostic and keeps an unexplained residual (Section V-G). The profiling attribution names a subsystem, not a function. The two inconclusive primary cells were underpowered; Power-PUT repeated them on x86 only, so on ARM 64 KiB PUT at 128 workers and 64 KiB GET at 500 workers remain inconclusive. Both controls, Control-Manifest and Control-Binding, ran on the x86 client only: the ARM client uses the same binding and shows a similar deficit, but the attribution to the binding is not tested there. The line we draw between a saturated and an unsaturated Python cell rests on a difference of a few tenths of a core, was drawn after the data and was not pre-registered; anomaly A3 may be a consequence of drawing it too high.

Threats to external validity are the limits of what the data supports. Most of the study uses one Python client with liboqs through a binding, and Control-Binding shows how much that instrument can shape a result: its one client-bound deficit was produced by the binding. The native reader removes the interpreter but covers only the GET path of three modes at one object size, uses different cryptographic implementations, and reaches saturation only when confined to one core, which is an artificial condition; a production client saturating many cores may behave differently. The baseline set is narrow: no ML-KEM-1024, no RSA-3072, and no remote key-service wrap, which is a common deployment path. MinIO ran as a single node without erasure coding or replication, in a community edition that is no longer developed; a distributed deployment has a different write path and may have a different ceiling and knee. Both hosts sat in one placement group with a sub-millisecond round trip; a wide-area client would be dominated by the network. All runs used AWS, in two regions, with one server instance type. liboqs is intended for prototyping, not production.

Threats to construct validity concern what the measures mean. Throughput equivalence within ± 5 % is an engineering criterion, not a statement that two modes are identical; a ratio of 0.960 with a narrow interval is formally equivalent and still a real difference. The CPU accounting is an identity on the runs it describes; in the native reader it holds by construction; its one advance prediction missed by up to 0.7 points and lay outside both intervals. The statement that a throughput cost appears only at or near client saturation is contradicted by

anomaly A2 if "saturation" is read strictly, and the Python client's process CPU and the native reader's are not measured the same way (thread CPU at 500 workers against process CPU on one core), so their per-request figures should not be compared as like for like. The exploratory verdict counts are not corrected for multiplicity. Runs last 20 s; slower effects such as compaction or long-term memory growth are not observed.

The following have not been run, and the paper's claims are limited accordingly. (1) *A full native client:* PUT, the remaining modes and several object sizes, saturating many cores, with a second ML-KEM implementation so that library and language can be separated. (2) *A manifest control in the native reader*, to split its ML-KEM cost into key recovery and manifest handling by measurement and not by a separate timing. (3) *An HPKE-conformant envelope*, to confirm that the per-object costs transfer. (4) *Wider baselines:* ML-KEM-1024, RSA-3072 and a remote key-service wrap. (5) *A distributed or second S3 implementation*, and runs of 10 to 30 minutes, for the store-side results. (6) *The two controls on the ARM client.* (7) *Multi-recipient objects*, which would need a key-committing construction.

VII. Discussion and Conclusion

A practitioner learns more from these measurements than the back-of-envelope estimate gives. That tens of microseconds of KEM are small against milliseconds of storage is true but incomplete. Three things could not have been read off that estimate. First, the regime in which a cost appeared is narrow and identifiable: a client at or near CPU saturation, with the size of the cost given by client CPU per request. Second, the first cause we suspected was wrong. The 1,088-byte ciphertext and its handling explain 17 % of the x86 Python client's deficit; the rest came from a binding that crosses into native code four times per decapsulation. Third, the size of the effect depends on the library: the ML-KEM operation was cheaper than an X25519 exchange with liboqs called directly and dearer in Go's standard library, and the throughput cost under saturation ranged accordingly from equivalence within ± 2 % to 9.5 %. In no saturated cell was ML-KEM faster than X25519.

For migration planning, the CPU accounting of Section IV-C gives a short procedure. Measure client CPU per request for the current and the candidate mode on the real workload and client. If clients are well below CPU saturation, our data gives no reason to expect a change in throughput, with the reservation of anomaly A2. If they are saturated, expect throughput to change by about the inverse ratio of the two CPU figures: 4.0–4.9 % in our Python client, 9.5 % in our native reader on one core, and equivalence within ± 2 % with a one-call binding on x86. A measurement on one client stack does not transfer to another.

For library authors the lesson concerns bindings. A binding that allocates an object and makes several foreign calls per operation can cost more under thread contention than the cryptography it wraps. In our x86 client, holding a handle per process and making one call per operation removed 99 % of the deficit (90 % CI 92–105 %); whether that call released the interpreter lock made no measurable difference.

The cost that does not depend on the implementation is ML-KEM's 1,088-byte ciphertext: 1,056 bytes more per object than an X25519 key, or 1,412 more in a JSON/base64 manifest, which puts such a manifest at or above the 2 KB that Amazon S3 allows for user-defined metadata (Section V-E). For hybrids there is in addition a client-CPU increment under saturation. RSA-4096 is the poor choice: its private-key operation makes small-object reads CPU-bound at 0.42–0.46 of ML-KEM's throughput in the Python client.

On the capacity of the store, the open-loop measurements are reported in the supplementary material because they describe the store and not post-quantum cryptography. Their result is that mean response time doubles at 0.68–0.81 of peak closed-loop throughput, in line with the known gap between closed- and open-loop behaviour [15]; the exploratory campaign found that knee in the same place for the symmetric-only mode and for ML-KEM.

In conclusion, we evaluated per-object post-quantum key establishment on an S3-compatible store with a Python client on x86 and ARM and with a native Go reader, against a single-node MinIO server on AWS. In these configurations ML-KEM lowered throughput against X25519 only where the client was at or near CPU saturation, with one unexplained exception in the native reader (Section V-G). In the one saturated cell of the pre-registered family the Python client was 4.0 % (x86) and 4.9 % (ARM) slower; two pre-registered controls on x86 attribute that deficit to the library binding and 17 % of it to the larger manifest, and with one foreign call per operation ML-KEM was equivalent to X25519 within ± 2 %. A native reader saturated on one core was 9.5 % slower with ML-KEM and 17.4 % with X-Wing; a slower key recovery in its library accounts for about half of the extra CPU for ML-KEM and about three quarters for X-Wing. In the saturated cells the throughput loss matched the extra client CPU per request, which makes CPU per request the quantity to measure before a migration. The size of the cost depends on the client implementation. The cost that every implementation pays is storage: 1,056 bytes more per object than X25519.

These results describe two clients, one single-node store and one cloud provider, and the native reader covers the GET path only. They should not be extended to distributed object stores, other S3 implementations, other libraries or wide-area deployments without new measurements (Section VI).

Data and Code Availability

Code, configurations, the pre-registration, per-run summaries, the raw-data validator, the native reader (`native/`) and the analysis scripts that generate every figure, table and number in this paper (`analysis/summarize.py`, `final_analysis.py`, `make_paper_assets.py`, `revision_assets.py`, `audit_assets.py`) are in the project repository, organized following ACM artifact guidelines [52]; the configuration files record the random seed of every experiment, and the

repository holds an export of the hosting service's push log. The repository is public at [link withheld for double-anonymised review; it is given on the title page]; it has no persistent identifier. The raw request records (about 11 GB compressed) are not in the repository because of their size and are available from the corresponding author on request.

References

[1] National Institute of Standards and Technology, "Module-lattice-based key-encapsulation mechanism standard," NIST, Federal Information Processing Standards Publication FIPS 203, Aug. 2024.

[2] Amazon Web Services, "Amazon S3 Encryption Client concepts," Amazon S3 Encryption Client Developer Guide, https://docs.aws.amazon.com/amazon-s3-encryption-client/latest/developerguide/concepts.html, 2026.

[3] Microsoft, "Client-side encryption for blobs," Azure Storage documentation, https://learn.microsoft.com/en-us/azure/storage/blobs/client-side-encryption, 2026.

[4] Open Quantum Safe Project, "liboqs: C library for prototyping and experimenting with quantum-resistant cryptography, version 0.16.0," https://github.com/open-quantum-safe/liboqs, 2026.

[5] N. Rodriguez-Alvarez, "Performance and storage analysis of CRYSTALS-Kyber as a post quantum replacement for RSA and ECC," arXiv:2508.01694, 2025.

[6] E. D. Demir, B. Bilgin, and M. C. Onbasli, "Performance analysis and industry deployment of post-quantum cryptography algorithms," arXiv:2503.12952, 2025.

[7] J. Lopez, V. Cadena, and M. S. Rahman, "Evaluating post-quantum cryptographic algorithms on resource-constrained devices," arXiv:2507.08312, 2025.

[8] C. Paquin, D. Stebila, and G. Tamvada, "Benchmarking post-quantum cryptography in TLS," in *Post-Quantum Cryptography (PQCrypto 2020)*, 2020, Cryptology ePrint Archive, Paper 2019/1447.

[9] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, "Post-quantum authentication in TLS 1.3: A performance study," in *Network and Distributed System Security Symposium (NDSS)*, 2020.

[10] M. Sosnowski, F. Wiedner, E. Hauser, L. Steger, D. Schoinianakis, S. Gallenmüller, and G. Carle, "The performance of post-quantum TLS 1.3," in *Companion of the 19th International Conference on emerging Networking EXperiments and Technologies (CoNEXT)*, 2023, pp. 19–27.

[11] A. A. Giron, J. P. A. do Nascimento, R. Custódio, L. P. Perin, and V. Mateu, "Post-quantum hybrid KEMTLS performance in simulated and real network environments," in *Progress in Cryptology – LATINCRYPT 2023*, ser. Lecture Notes in Computer Science, 2023, pp. 293–312.

[12] F. Gadban and J. Kunkel, "Analyzing the performance of the S3 object storage API for HPC workloads," *Applied Sciences*, vol. 11, no. 18, p. 8540, 2021.

[13] E. Henziger and N. Carlsson, "Delta encoding overhead analysis of cloud storage systems using client-side encryption," in *Proc. 11th IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, 2019.

[14] O. Ozeri, D. Harnik, and E. Ofer, "Benefits of encryption at the storage client," in *Proc. 16th ACM International Conference on Systems and Storage (SYSTOR)*, 2023, p. 149.

[15] B. Schroeder, A. Wierman, and M. Harchol-Balter, "Open versus closed: A cautionary tale," in *Proc. 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2006, pp. 239–252.

[16] M. Barbosa, D. Connolly, J. D. Duarte, A. Kaiser, P. Schwabe, K. Varner, and B. Westerbaan, "X-Wing: The hybrid KEM you've been looking for," *IACR Communications in Cryptology*, vol. 1, no. 1, 2024.

[17] D. Connolly, P. Schwabe, and B. Westerbaan, "X-Wing: General-purpose hybrid post-quantum KEM," IETF Internet-Draft draft-connolly-cfrg-xwing-kem-11, Sep. 2026.

[18] R. Barnes, K. Bhargavan, B. Lipp, and C. A. Wood, "Hybrid public key encryption," RFC 9180, Feb. 2022.

[19] R. Barnes and D. Connolly, "Post-quantum and post-quantum/traditional hybrid algorithms for HPKE," IETF Internet-Draft draft-ietf-hpke-pq-05 (work in progress), Jul. 2026.

[20] F. Valsorda and age contributors, "age v1.3.0: post-quantum recipients," Software release, https://github.com/FiloSottile/age/releases/tag/v1.3.0, Dec. 2025.

[21] F. Giacon, F. Heuer, and B. Poettering, "KEM combiners," in *Public-Key Cryptography – PKC 2018*, ser. Lecture Notes in Computer Science, 2018, cryptology ePrint Archive, Paper 2018/024.

[22] N. Bindel, J. Brendel, M. Fischlin, B. Goncalves, and D. Stebila, "Hybrid key encapsulation mechanisms and authenticated key exchange," in *Post-Quantum Cryptography – PQCrypto 2019*, ser. Lecture Notes in Computer Science, vol. 11505, 2019.

[23] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," RFC 5869, 2010.

[24] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST, Tech. Rep. SP 800-38D, 2007.

[25] K. Moriarty, B. Kaliski, J. Jonsson, and A. Rusch, "PKCS #1: RSA cryptography specifications version 2.2," RFC 8017, 2016.

[26] A. Langley, M. Hamburg, and S. Turner, "Elliptic curves for security," RFC 7748, 2016.

[27] MinIO, Inc., "MinIO object storage server, release RELEASE.2025-10-15T17-29-55Z (repository archived April 2026)," https://github.com/minio/minio, 2025.

[28] E. Kret and R. Schmidt, "The PQXDH key agreement protocol," Signal specification, revision 3, https://signal.org/docs/specifications/pqxdh/, May 2023.

[29] Apple Security Engineering and Architecture, "iMessage with PQ3: The new state of the art in quantum-secure messaging at scale," Apple Security Research blog, https://security.apple.com/blog/imessage-pq3/, Feb. 2024.

[30] L. F. Prikutse, R. E. Turkson, A.-S. Yussiff, A.-L. Yussiff, and M. G. Asante-Mensah, "A commitment-based hybrid post-quantum cryptographic model for multi-file cloud storage," arXiv:2608.13138, 2026.

[31] M. S. B. Junaid, "A hybrid post-quantum encryption architecture with self-hosted key management for SME cloud data protection," arXiv:2608.26777, Aug. 2026.

[32] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Kyber: A CCA-secure module-lattice-based KEM," in *Proc. IEEE European Symposium on Security and Privacy (EuroS&P)*, 2018, pp. 353–367.

[33] P. Schwabe, D. Stebila, and T. Wiggers, "Post-quantum TLS without handshake signatures," in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2020, pp. 1461–1480.

[34] Amazon Web Services, "Reducing the cost of SSE-KMS with Amazon S3 Bucket Keys," Amazon S3 User Guide, https://docs.aws.amazon.com/AmazonS3/latest/userguide/bucket-key.html, 2026.

[35] ——, "Data key caching details," AWS Encryption SDK Developer Guide, https://docs.aws.amazon.com/encryption-sdk/latest/developer-guide/data-caching-details.html, 2026.

[36] P. Leitner and J. Cito, "Patterns in the chaos: A study of performance variation and predictability in public IaaS clouds," *ACM Transactions on Internet Technology*, vol. 16, no. 3, pp. 15:1–15:23, 2016.

[37] A. Uta, A. Custura, D. Duplyakin, I. Jimenez, J. Rellermeyer, C. Maltzahn, R. Ricci, and A. Iosup, "Is big data performance reproducible in modern cloud networks?" in *Proc. 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020.

[38] A. Maricq, D. Duplyakin, I. Jimenez, C. Maltzahn, R. Stutsman, and R. Ricci, "Taming performance variability," in *Proc. 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018, pp. 409–425.

[39] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015, pp. 73:1–73:12.

[40] T. Kalibera and R. Jones, "Rigorous benchmarking in reasonable time," in *Proc. International Symposium on Memory Management (ISMM)*, 2013, pp. 63–74.

[41] D. J. Schuirmann, "A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability," *Journal of Pharmacokinetics and Biopharmaceutics*, vol. 15, no. 6, pp. 657–680, 1987.

[42] D. Lakens, "Equivalence tests: A practical primer for t tests, correlations, and meta-analyses," *Social Psychological and Personality Science*, vol. 8, no. 4, pp. 355–362, 2017.

[43] Y. Dodis, P. Grubbs, T. Ristenpart, and J. Woodage, "Fast message franking: From invisible salamanders to encryptment," in *Advances in Cryptology – CRYPTO 2018*, ser. Lecture Notes in Computer Science, vol. 10991, 2018, pp. 155–186.

[44] J. Len, P. Grubbs, and T. Ristenpart, "Partitioning oracle attacks," in *Proc. 30th USENIX Security Symposium*, 2021, pp. 195–212.
[45] A. Albertini, T. Duong, S. Gueron, S. Kölbl, A. Luykx, and S. Schmieg, "How to abuse and fix authenticated encryption without key commitment," in *Proc. 31st USENIX Security Symposium*, 2022, pp. 3291–3308.
[46] V. T. Hoang, R. Reyhanitabar, P. Rogaway, and D. Vizár, "Online authenticated-encryption and its nonce-reuse misuse-resistance," in *Advances in Cryptology – CRYPTO 2015*, ser. Lecture Notes in Computer Science, vol. 9215, 2015, pp. 493–517.
[47] Open Quantum Safe Project, "liboqs-python: Python 3 bindings for liboqs, version 0.16.0.1," https://github.com/open-quantum-safe/liboqs-python, 2026.
[48] Python Cryptographic Authority, "cryptography, version 50.0.1," https://cryptography.io, 2026.
[49] P. M. Dixon, P. F. Saint-Maurice, Y. Kim, P. Hibbing, Y. Bai, and G. J. Welk, "A primer on the use of equivalence testing for evaluating measurement agreement," *Medicine & Science in Sports & Exercise*, vol. 50, no. 4, pp. 837–845, 2018.
[50] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
[51] Amazon Web Services, "Working with object metadata," Amazon S3 User Guide, https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingMetadata.html, 2026.
[52] Association for Computing Machinery, "Artifact review and badging, version 1.1," https://www.acm.org/publications/policies/artifact-review-and-badging-current, 2020.